CHAOS OS

Break it. Learn from it. Build infrastructure that survives it.

The complete project guide for the team: every component, every result, and how to explain it.

Team members: [Name 1] · [Name 2] · [Name 3] · [Name 4]

Course / department: [Course name] · [Department, College]

Project guide: [Guide name]

Date: 4 November 2026

Repository: https://github.com/Prarthan-B-P/chaos-os

Every number in this guide was measured in the lab and is generated from the experiment database.

Contents

1. The big picture 5
1.1 The problem we solve 5
1.2 Elevator pitches (memorise one) 5
1.3 The project at a glance 6
1.4 The five headline results 6
1.5 How to use this guide 6
1.6 Glossary (the 40 words you must know) 7
2. Background concepts, from zero 9
2.1 DevOps and CI/CD 9
2.2 Containers, Docker and Docker Compose 9
2.3 Chaos engineering 9
2.4 SRE: SLIs, SLOs, error budgets and burn rates 10
2.5 Observability: metrics, dashboards, alerts 10
2.6 Infrastructure as code with Ansible 10
2.7 Jenkins 11
2.8 Contextual bandits and LinUCB 11
2.9 LLMs, tool calling and agents 11
2.10 Safety: guardrails, shields and human-in-the-loop 11
2.11 DevSecOps and software supply chain 12
2.12 Reverse proxies, HTTPS and tunnels 12
3. Architecture: every part and why it exists 13
3.1 The system under test: a three-tier web shop 13
3.2 Every container 13
3.3 The three main data flows 14
3.4 Where things live in the repository 15
4. How an experiment works 16
4.1 The life of one trial 16
4.2 Symptoms: what a policy is allowed to see 16
4.3 The recovery definition (be precise about this) 16
4.4 The seven failures 17
4.5 The six recovery actions (Ansible playbooks) 17
4.6 The safety validator 18
5. The responders: who decides what to do 19
5.1 Rule-based runbook (the classic approach) 19
5.2 Expert heuristic (hand-written rules) 19
5.3 LinUCB contextual bandit (machine learning) 19

5.4 LLM policy (gpt-4o-mini) 20
5.5 LLM agent with tools (agentic AI) 20
5.6 The ablation variants 20
5.7 Adversarial (red-team) policy 21
6. On-call mode: healing failures nobody announced 22
6.1 The story of one incident, step by step 22
6.2 Why the agent re-checks every page 23
6.3 Human-in-the-loop approvals 23
6.4 The chaos side: Break it, chaos monkey, rollback 24
6.5 The scorecard: how we prove it works 24
6.6 Error budget on the dashboard 24
7. The DevOps machinery 26
7.1 The Jenkins pipeline, stage by stage 26
7.2 The resilience gate and the demo bug 27
7.3 Alerts are code, and code is tested 27
7.4 DevSecOps: the security scan 27
7.5 Deployment 28
7.6 The evidence dashboard, page by page 29
7.7 Generative AI features (gpt-4o-mini only) 29
8. Hands-on: running and reproducing everything 31
8.1 Start the lab 31
8.2 Run experiments from the command line 31
8.3 On-call mode 31
8.4 CI, alert-rule tests and the security scan 32
8.5 Regenerate the paper's figures and this guide 32
8.6 Share it online 32
8.7 Troubleshooting 32
9. Results 33
9.1 Result 1: every failure needs a different fix 33
9.2 Result 2: policies on training and held-out failures 34
9.3 Result 3: the bandit learns from experience 35
9.4 Result 4: the LLM ablation (tools vs. advice) 35
9.5 Result 5: on-call mode heals unannounced failures 37
9.6 Result 6: safety, CI and security 38
10. Defending the project 39
10.1 Limitations (say these yourself) 39
10.2 Future work 39
10.3 The live demo (about 7 minutes) 39

10.4 Viva questions with model answers 40
10.5 Who explains what (suggested split for four people) 45
11. Code tour: where each idea lives 46
11.1 The experiment engine (controller/chaosctl) 46
11.2 Learning and AI 46
11.3 On-call mode 47
11.4 DevOps 48
12. Appendix: the key files, annotated 49
12.1 safety-policy.yml: the safety rules 49
12.2 observability/prometheus/rules/slo.yml: SLO and alerts 49
12.3 observability/alertmanager/alertmanager.yml: routing pages to the agent 50
12.4 resilience-gate.yml: what every release must survive 51
12.5 ansible/playbooks/restart_service.yml: one recovery action 51
12.6 Jenkinsfile: the stages 52
12.7 .trivyignore.yaml: accepted security risks, each with a reason 54

1. The big picture

CHAOS OS is a resilience lab. It breaks a small online shop on purpose, lets different "responders" try to fix it, measures exactly how long every fix takes, keeps every action safe, learns which fix works best, and stops a bad release before it ships.

If you remember one sentence, remember this one: **CHAOS OS turns "we think our system recovers from failures" into a measured, repeatable, automatically tested fact.**

1.1 The problem we solve

Every real system fails: a cache server crashes, a database becomes slow, the network drops packets, an application process dies. Teams usually write a *runbook* ("if X breaks, restart Y") and hope it works. Three things go wrong in practice:

- **Nobody measures whether the runbook is right.** "Restart it" is the default answer, but our data shows it is the wrong answer for most failures.
- **Automation can make things worse.** A script that restarts things in a loop, or touches the wrong service, turns a small incident into a big one. Automation needs guardrails.
- **Releases are not tested against failures.** Unit tests check that code works when everything is healthy. They do not check that the system still *recovers* when something breaks.

CHAOS OS attacks all three: it measures every fix against every failure, puts a safety validator (and a human, for risky actions) between any decision and its execution, and runs real failures in the CI/CD pipeline so a release that cannot recover is blocked.

1.2 Elevator pitches (memorise one)

> **THE 30-SECOND VERSION**
>
> "We built a lab that injects real failures into a web shop under live traffic. Five kinds of responder (a classic runbook, expert rules, a machine-learning bandit, an LLM, and an LLM agent with tools) try to fix each failure; a safety validator checks every action first. We measure recovery for every single request, learn which fix is best, run an always-on AI on-call engineer that heals failures nobody announced, and make Jenkins refuse any release that cannot recover."
>
> **THE 2-MINUTE VERSION (ADD THESE AFTER THE 30 SECONDS)**
>
> "Our first finding is that **there is no universal fix**: each failure has a different best action, and the classic 'restart it' runbook fails exactly where restarting is the wrong answer.
>
> Second, a **contextual bandit** (a light-weight learning algorithm) learned the right fix from logged experiments, without anyone writing rules, and it **generalised to a failure it had never seen** (a Redis network cut).
>
> Third, we tested **gpt-4o-mini** carefully with an ablation study: one paragraph of general SRE advice changed its behaviour far more than tools did, for better on network faults and for worse on crashes; tools alone only added seconds, but tools plus advice was the most robust combination.

> Fourth, the lab now runs like production: **SLOs, error budgets and burn-rate alerts** page an **on-call agent** that confirms the problem, fixes it through the safety validator, asks a human before anything risky, and writes the post-mortem.
>
> And it is all **DevOps end to end**: Docker Compose, Ansible, Jenkins with resilience, security and on-call gates, Prometheus, Grafana, Alertmanager, and one-command HTTPS deployment."

1.3 The project at a glance

What	Number
Containers in the running lab	11 (shop, data, fault injection, observability, agent, dashboard)
Failure scenarios	7 (5 training + 2 held out, never used for training)
Recovery actions	6 (none, restart Redis / Postgres / API, cache bypass, warm cache)
Responders (policies) compared	5 main + 2 ablation variants + 1 red-team policy
Experiment trials recorded	182 (every request of every trial timed)
LLM ablation trials (2 x 2 design)	56
Unannounced faults in the on-call campaign	21 (18 healed)
Automated tests	45 unit tests + 4 alert-rule tests + 6 gate checks + 1 on-call drill
Jenkins pipeline stages	10 (build to teardown)
Lines of code (non-blank)	about 6,100 across 132 files
Unsafe actions executed	0

1.4 The five headline results

1. **No single fix works everywhere.** Each of the 5 training failures has a different best action. Choosing the wrong one costs hundreds to over a thousand failed requests (Figure in Chapter 9).
2. **The learned bandit matches expert rules without hand-written rules** and generalises to the unseen Redis network partition (3/3 recovered, 0.8 s), where the classic runbook fails 0/3.
3. **For the LLM, advice and evidence interact.** Generic SRE guidance replaced gpt-4o-mini's "restart first" habit with a "bypass first" habit (Redis latency 50 s → 3 s, but a crashed database was never restarted); tools alone only added seconds; tools plus guidance handled both kinds of fault.
4. **The on-call agent heals failures nobody announced**, end to end: alert, confirmation, safe fix, human approval when needed, post-mortem.
5. **Safety held: zero unsafe actions** in every experiment, including deliberate red-team attacks, and the CI gate blocks a one-character bug that unit and smoke tests miss.

1.5 How to use this guide

- **Chapters 2-3** teach the background (DevOps, SRE, chaos engineering, AI) from zero. Read them first if a term is new.

- **Chapters 4-8** explain every part of the system: what it does, why it exists, and where the code is.
- **Chapter 9** has every result, with the numbers you should quote.
- **Chapter 10** is for defending the project: limitations, the demo script, about 50 likely teacher questions with answers, and a split of who explains what.
- **Chapter 11** is a code tour (which file and function to open for each idea) and **Chapter 12** reprints the key configuration files with notes.
- Look for the coloured boxes: **orange = say this to the teacher**, **grey = remember this**, **blue = likely question**, **yellow = common mistake**.

1.6 Glossary (the 40 words you must know)

Term	Meaning in one line
Chaos engineering	Breaking a system on purpose, in a controlled way, to learn how it fails and recovers.
Fault / failure scenario	One specific way we break the shop (e.g. kill Redis, slow down Postgres).
Fault injection	The act of causing the fault. We use Docker (kill containers) and Toxiproxy (network).
Toxiproxy	A proxy between the API and Redis/Postgres that can add latency or cut the connection.
Recovery action	One fix we can apply, implemented as an Ansible playbook.
Policy / responder	The decision-maker that picks a recovery action from the symptoms.
Symptoms	What an operator can observe: error rate, p95 latency, API up?, Redis/Postgres reachable + probe ms.
Trial	One experiment: reset, warm up, inject, detect, remediate, measure.
Recovery time	Seconds from fault injection until traffic is healthy again (strict definition in 4.3).
p95 latency	95% of requests were faster than this value.
Safety validator	Code that checks every proposed action against safety-policy.yml before it runs.
Allowlist	The explicit list of things that are permitted; anything else is refused.
Human-in-the-loop	A person must approve high-impact actions (here: restart_postgres in on-call mode).
Red team	A deliberately hostile tester; our adversarial policy proposes unsafe actions on purpose.
Contextual bandit	A learning algorithm that picks an action based on context and learns from rewards.
LinUCB	The specific bandit algorithm we use: one linear model per action plus an uncertainty bonus.
Reward	How good an action was: faster healing = higher; minus a safety cost for risky actions.
Held-out scenario	A failure never used for designing or training, used only to test generalisation.
LLM	Large language model; we use OpenAI gpt-4o-mini only.
LLM agent / tool calling	An LLM that can call functions (query metrics, read logs) before deciding.

Term	Meaning in one line
Ablation study	Turning features on/off one at a time to see which one causes an effect.
SRE	Site Reliability Engineering: running services with engineering discipline (Google's practice).
SLI	Service Level Indicator: what we measure (share of good user requests).
SLO	Service Level Objective: the target (99% of requests good).
Error budget	The allowed bad part: 1% of requests. Spending it too fast is an emergency.
Burn rate	How fast the error budget is being spent; 1x = exactly on budget, 14.4x = paging speed.
Page / ticket	Page = urgent alert that wakes someone (here the agent); ticket = look at it later.
Alertmanager	Prometheus' alert router: groups, de-duplicates and sends alerts (here: webhook to the agent).
Synthetic users	A small service that sends steady fake customer traffic and measures the SLI.
On-call agent	Our always-on program that responds to pages, like a human on-call engineer.
MTTD / MTTR	Mean (we report median) time to detect / to recover.
Post-mortem	A blameless write-up of an incident: what happened, what was done, what we learn.
Docker / container	A packaged, isolated process with everything it needs to run.
Docker Compose	A file that describes many containers and starts them together.
Ansible / playbook	Automation tool; a playbook is a YAML list of steps (here: our recovery actions).
CI/CD / Jenkins	Automatic build-test-release pipeline; Jenkins is the server that runs it.
Resilience gate	Pipeline stage that injects real faults and fails the build if recovery is too slow.
DevSecOps	Security checks built into the pipeline (secrets, vulnerable dependencies, image CVEs, SBOM).
SBOM	Software Bill of Materials: the list of every package inside an image.
Reverse proxy (Caddy)	The single public entry point that adds HTTPS and a login in front of the app.

2. Background concepts, from zero

This chapter explains every idea the project is built on. Each section ends with how that idea appears in CHAOS OS, so you can connect theory to our code.

2.1 DevOps and CI/CD

DevOps is the practice of developers and operations working as one team, automating the path from code to production and learning from how the system behaves in production. Its core loop is: plan, code, build, test, release, deploy, operate, monitor, and feed what you learn back into planning.

CI (Continuous Integration) means every change is built and tested automatically. **CD (Continuous Delivery/Deployment)** means a change that passes every check is ready to ship (or ships automatically). The pipeline is a series of *gates*; if any gate fails, the change stops.

> **IN CHAOS OS**
> Jenkins runs our pipeline on every branch. Besides normal gates (build, unit tests, smoke test), it has three unusual ones: a **security scan**, a **resilience gate** that injects real failures, and an **on-call drill** that checks the AI agent can heal an unannounced failure.

2.2 Containers, Docker and Docker Compose

A **container** packages a program with everything it needs (libraries, config) so it runs the same on any machine. **Docker** builds images and runs containers. **Docker Compose** describes a whole multi-container application in one YAML file (`docker-compose.yml`): which images, which network, which volumes, which health checks, which order to start in.

Two Compose ideas matter for us: **health checks** (`docker compose up --wait` waits until every service is healthy) and **project names** (every lab gets a name such as `chaosos` or `chaosos-ci-2`, so several labs can run side by side on one machine without touching each other).

> **IN CHAOS OS**
> The lab is 11 containers in one Compose project. We also use overlay files: `docker-compose.override.yml` adds local ports, `deploy/docker-compose.prod.yml` adds the HTTPS proxy for servers, and `deploy/docker-compose.tunnel.yml` shares the lab through a Cloudflare tunnel.

2.3 Chaos engineering

Chaos engineering was popularised by Netflix (Chaos Monkey randomly terminated servers in production). The principle: *you do not know how your system fails until you make it fail*. A chaos experiment has a **steady-state hypothesis** ("traffic is healthy"), a **fault** ("kill the cache"), a controlled **blast radius**, and a **measurement** of what happened.

Most chaos tools (Chaos Mesh, LitmusChaos, Gremlin) only *inject* faults. CHAOS OS goes further: it compares many *recovery strategies* for each fault, learns which one is best, and turns the results into a release gate.

2.4 SRE: SLIs, SLOs, error budgets and burn rates

Site Reliability Engineering (SRE) is Google's way of running services. Its key tools:

- **SLI (indicator):** a measurement of user experience. Ours: the share of user requests that return 200 OK within 300 ms.
- **SLO (objective):** the target for the SLI. Ours: 99% of requests good.
- **Error budget:** what the SLO allows to be bad: 1% of requests. At our steady 2 requests/s that is 172,800 requests/day, so 1,728 bad requests per day are "allowed".
- **Burn rate:** how fast the budget is being spent. If 1% of requests are bad, the burn rate is 1x (exactly on budget). If 14.4% are bad, it is 14.4x: at that speed a 30-day budget would be gone in 2 days.

Why 14.4x? It comes from the Google SRE Workbook: burning 2% of a 30-day budget in one hour equals a burn rate of 14.4 (0.02 x 720 hours). It is the standard threshold for an urgent page.

Multi-window alerting. A page fires only if the burn rate is high over a **long** window (it is a real problem, not a blip) **and** a **short** window (it is still happening). Our lab compresses time: 1 minute and 20 seconds instead of 1 hour and 5 minutes. A 5-second blip does not page; a real outage pages in about 10-15 seconds; the page stops soon after recovery.

> **WORKED EXAMPLE TO SAY OUT LOUD**
> "At 2 requests per second, a 1-minute window holds 120 requests. If Redis dies, every request fails. After about 9 seconds, 18 of the last 120 requests are bad: 15%, which is a burn rate above 14.4. The 20-second window is already at 45%. Both windows are over the line, so Prometheus fires the page."

2.5 Observability: metrics, dashboards, alerts

Prometheus collects numeric metrics by *scraping* an HTTP `/metrics` endpoint every few seconds (ours: every 5 s) and stores them as time series. **Recording rules** pre-compute useful series (our bad ratios over 20 s, 1 min, 10 min, 1 h). **Alerting rules** turn conditions into alerts.

Alertmanager receives alerts, groups and de-duplicates them, silences or inhibits them, and routes them to a receiver (email, chat, or in our case a **webhook** that wakes the on-call agent). **Grafana** draws dashboards from Prometheus and shows **annotations** (our experiments and the agent mark fault injection and fixes on the graphs).

2.6 Infrastructure as code with Ansible

Ansible automates operations with YAML **playbooks**: a list of tasks, each using a module (run a command, call a URL, assert a condition). It is *idempotent* and readable, which makes it auditable. Every recovery action in CHAOS OS is an Ansible playbook, and the playbooks enforce their own **service allowlist** (a second layer of safety under the validator). Ansible is also used to **provision a cloud server** in one command (`deploy/ansible/deploy.yml`).

2.7 Jenkins

Jenkins is an open-source automation server. A `Jenkinsfile` (pipeline as code) lists the stages. Our job is a **multibranch** pipeline defined as code (`jenkins/jobs/chaos-os-ci/config.xml`): Jenkins polls GitHub every 5 minutes and builds every branch. Jenkins runs in its own container and controls the host's Docker engine through the Docker socket ("Docker outside of Docker"), so each build starts its own isolated lab (`chaosos-ci-<executor>`) without any host ports.

2.8 Contextual bandits and LinUCB

A **multi-armed bandit** is a learning problem: you repeatedly choose one of several actions ("arms") and get a reward. A **contextual** bandit also sees a context (for us: the symptoms) and learns which action is best *for that context*. It is simpler than full reinforcement learning because each decision is judged on its own immediate reward.

LinUCB keeps, for every action, a small linear model that predicts reward from the context features, plus an **upper confidence bound** bonus that is large when the model is unsure. Choosing the highest "prediction + bonus" balances exploring unknown actions and exploiting good ones. At decision time in the lab we use the prediction only (greedy), because exploration happens in the logged experiments.

> **WHY A BANDIT AND NOT DEEP LEARNING?**
> We have a few hundred logged decisions, not millions. A linear bandit with 8 features learns from small data, trains in milliseconds, is explainable (you can read its weights), and runs without a GPU.

2.9 LLMs, tool calling and agents

A **large language model (LLM)** predicts text. We use **OpenAI gpt-4o-mini** only (cheap and fast). With **JSON mode** it answers in a fixed structure (`{"action": ..., "reason": ...}`). With **tool calling**, the model can ask the program to run a function (e.g. "query Prometheus", "read the Redis logs"), read the result, and continue: this loop of reasoning and acting is called an **agent** (the ReAct pattern). An agent can investigate before acting, but every extra call costs time.

> **COMMON MISTAKE**
> Do not say "the AI fixes the system". Say: "the LLM *proposes* an action; the safety validator decides whether it may run; Ansible runs it; and we measure whether it worked."

2.10 Safety: guardrails, shields and human-in-the-loop

Letting software act on production is dangerous. In safe reinforcement learning, a **shield** sits between the learner and the world and blocks unsafe actions. Our **safety validator** is such a shield: every proposal from every policy is checked against `safety-policy.yml` (approved environment, allowlist, no repeats, action budget, approval for high-impact actions). In on-call mode, high-impact actions wait for a **human** in the dashboard; if nobody answers in 60 s, the action is **not** run (fail safe).

2.11 DevSecOps and software supply chain

- **Secret scanning** (gitleaks): finds passwords and API keys committed to code or hidden in git history.
- **Dependency scanning** (Trivy fs): matches pinned Python/npm packages against vulnerability databases (CVEs).
- **Misconfiguration scanning** (Trivy, IaC): flags risky Dockerfiles, e.g. containers running as root.
- **Image scanning** (Trivy image): finds vulnerable OS packages and libraries inside built images.
- **SBOM** (CycloneDX): a machine-readable list of everything inside each image, archived per build.

2.12 Reverse proxies, HTTPS and tunnels

A **reverse proxy** is the single front door of a deployment. Ours, **Caddy**, gets free HTTPS certificates automatically, asks for a login (bcrypt-hashed password), adds security headers, and forwards to the dashboard and Grafana. Nothing else is reachable from the internet. Without a server, a **Cloudflare quick tunnel** gives the laptop a temporary public HTTPS address by connecting outwards, so no router port is opened.

3. Architecture: every part and why it exists

CHAOS OS has three layers: the **system under test** (a small online shop), the **observability and response** layer (metrics, alerts, the on-call agent), and the **control, safety and evidence** layer (experiments, policies, the safety validator, Ansible, results, the dashboard and gpt-4o-mini).

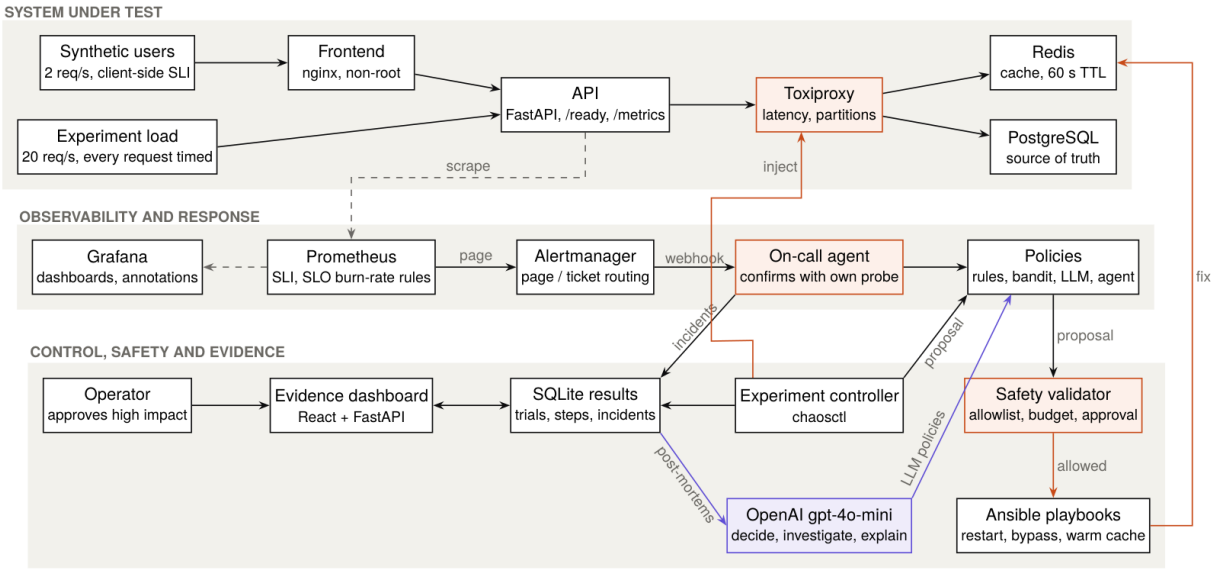


Figure 1. CHAOS OS architecture. Orange boxes are where faults and fixes enter the system; the purple box is the only part that talks to OpenAI. Faults are also injected from the dashboard's Break-it button and the chaos monkey.

3.1 The system under test: a three-tier web shop

The shop is deliberately simple so that every failure is understandable: a product catalogue of 1,000 items, read through a **cache-aside** pattern.

1. A request `GET /items/{id}` reaches the **API** (FastAPI, Python).
2. The API first asks **Redis** (the cache). Hit: answer immediately (about 1-2 ms).
3. Miss: the API reads **PostgreSQL** (the source of truth), stores the item in Redis for 60 s, and answers.
4. If Redis errors (down or slower than its 500 ms timeout), the API returns **503** unless **cache bypass** is on, in which case every read goes straight to Postgres.
5. If Postgres errors or its connection pool (10 connections, 2 s timeout) is exhausted, the API returns 503.

Important: the API reaches Redis and Postgres **through Toxiproxy**. That lets us add latency or cut the network without touching Redis or Postgres themselves, exactly like a real network problem.

3.2 Every container

Service	Technology	What it does	Why we need it
frontend	nginx (unprivileged, port 8080)	Serves the shop page; proxies /api/ to the API	A realistic entry point; the smoke test goes through it

Service	Technology	What it does	Why we need it
api	FastAPI + uvicorn, Python 3.12	Shop logic: /items, /health, /ready, /metrics, admin endpoints for fixes	The service whose recovery we measure
redis	Redis 7.2	Cache with 60 s TTL	Cache failures are common and subtle
postgres	PostgreSQL 16	Item database (1,000 rows)	Source of truth; slow or dead DB is a classic incident
toxiproxy	Shopify Toxiproxy 2.9	Network proxy for Redis and Postgres	Injects latency and partitions
traffic	Python, httpx, prometheus-client	Synthetic users: 2 req/s; counts good / slow / error	The SLI the alerts are built on
prometheus	Prometheus 2.53	Scrapes metrics every 5 s; SLO recording + alert rules	Monitoring and alerting brain
alertmanager	Alertmanager 0.27	Routes pages to the agent's webhook	Production-style alert routing
grafana	Grafana 11.1	Service-health and SLO dashboards with annotations	Visual evidence
oncall	chaosctl (Python) in the controller image	The on-call agent: webhook receiver + remediation	Heals unannounced failures
dashboard	React 18 + Vite 6 + Recharts; FastAPI	Evidence dashboard, approvals, chaos controls, gen AI	Shows everything; human-in-the-loop
controller (tool)	chaosctl CLI	Runs experiments, training, reports, gate, drill	The experiment engine
ansible (tool)	ansible-core 2.21.6	Runs recovery playbooks	Infrastructure as code for fixes
jenkins (separate)	Jenkins LTS + Docker CLI	CI/CD pipeline	Gates every release

3.3 The three main data flows

Flow A: an experiment (we know what we broke)

The **controller** resets the lab, sends 20 requests per second, injects a fault, detects the outage from its own traffic, asks a **policy** for an action, checks it with the **safety validator**, runs it with **Ansible**, waits up to 20 s for healing, repeats if needed (at most 3 actions), measures recovery, and saves the trial and every step to **SQLite**.

Flow B: on-call (nobody tells the agent anything)

The **synthetic users** see errors, **Prometheus** sees the SLO burning, **Alertmanager** sends a webhook, the **on-call agent** confirms with its own probe traffic, runs the same remediation loop (with a human approval step for high-impact actions), records an **incident**, and asks gpt-4o-mini for a post-mortem.

Flow C: a release (Jenkins)

Jenkins builds all images, runs unit tests and alert-rule tests, scans for secrets and vulnerabilities, starts an isolated lab, runs a smoke test, runs the **resilience gate** (6 real faults with deadlines) and the **on-call drill**, archives the reports, and tears the lab down.

3.4 Where things live in the repository

Path	Contents
app/api, app/frontend, app/db	The shop: API code, nginx config + page, database schema and seed data
app/traffic	Synthetic users (SLI)
chaos/toxiproxy	Toxiproxy proxy definitions (redis, postgres)
controller/chaosctl	The brain: load generator, analysis, experiments, safety, policies, bandit, LLM, agent, on-call, incidents, CLI
controller/tests	45 unit tests
ansible/playbooks	Recovery actions: restart*service, set*cache*bypass, warm*cache
observability/prometheus	Scrape config, SLO rules (rules/slo.yml) and their unit tests (tests/slo_test.yml)
observability/alertmanager	Routing to the agent; inhibition of tickets during pages
observability/grafana	Provisioned data source and dashboards
dashboard/server, dashboard/web	Dashboard back end (FastAPI) and front end (React)
safety-policy.yml	The safety rules (policy as code)
resilience-gate.yml	The CI resilience checks and deadlines (policy as code)
Jenkinsfile, jenkins/	The pipeline and the Jenkins image with the job defined as code
scripts/	Smoke test, security scan, on-call measurement campaign
deploy/	Production overlay (Caddy), Ansible server provisioning, laptop sharing tunnel
docs/	Demo runbook, this guide, the IEEE paper and its figure generator

4. How an experiment works

Every trial is a small, fully measured incident. The controller plays both the attacker (it injects the fault) and the on-call engineer (it runs the chosen responder), and records everything.

4.1 The life of one trial

1. **Reset** the lab to a known-good state: remove all Toxiproxy faults, start any stopped container, wait for `/ready`, switch cache bypass off, and flush Redis (every trial starts with a cold cache).
2. **Warm up** for 15 s at 20 requests/s. The 10 s before injection are the **baseline**; if the baseline is not healthy the trial is marked *invalid* and excluded.
3. **Inject** the fault.
4. **Detect** the outage from live traffic (the last 1-second window is unhealthy). If nothing breaks within 15 s, the fault had no impact.
5. **Remediation loop** (at most 3 executed actions): observe symptoms, ask the policy, check with the safety validator, run the Ansible playbook, wait up to 20 s for 3 healthy seconds. If it healed, stop; otherwise observe again and ask for the next action.
6. **Keep the load running** until recovery is confirmed or the timeout (60 s, 90 s for policy runs) expires.
7. **Reset** again and **save** the trial and every step (symptoms, action, verdict, timing, reward, the LLM's reason, the agent's investigation trail) to SQLite.

> **THE LOAD GENERATOR IS "OPEN MODEL"**
> It sends a request every 50 ms no matter how slow the answers are. A closed-model generator (wait for an answer before sending the next) would quietly send less traffic during an outage and hide the damage.

4.2 Symptoms: what a policy is allowed to see

Policies never see the name of the injected fault. They see only what an operator could observe:

Symptom	Source	Example during redis_latency
error_rate	load generator, last 1 s	1.00 (every request failing)
p95_ms	load generator, last 1 s	505 ms
api_up	API /ready answers?	true
redis*ok, redis*ms	/ready pings Redis	false, 501 ms (the 500 ms timeout)
postgres*ok, postgres*ms	/ready runs SELECT 1	true, 1.4 ms

4.3 The recovery definition (be precise about this)

Prometheus scrapes every 5 s, which is too coarse to measure recoveries of 1-2 s. So the load generator times every single request (timeout 2 s) and `controller/chaosctl/analysis.py` (unit-tested) decides:

- Time after injection is cut into **1-second windows** by request send time.
- A window is **healthy** when at least **99%** of its requests succeed and its **p95 latency is at most 300 ms**.
- The service has **recovered** once **5 consecutive healthy windows** follow the first unhealthy one.
- The **recovery time** is pinned below window resolution: the send time of the first request after the last bad request (failed or slower than 300 ms) before that healthy run.

> **WORKED EXAMPLE**
> Fault at t = 0. Requests fail from 0.05 s. The policy runs cache*bypass at 1.2 s; the last failing request was sent at 1.75 s; every window from 2 s to 7 s is healthy. Recovery time = send time of the first request after 1.75 s, i.e. 1.80 s. That is the 1.8 s you see for redis*down + cache_bypass.

4.4 The seven failures

Failure	How we inject it	What users see	Set
redis_down	docker kill the Redis container	503 errors (cache unavailable)	training
redis_latency	Toxiproxy adds 800 ms to Redis traffic (> the 500 ms timeout)	503 errors although Redis is running	training
postgres_latency	Toxiproxy adds 400 ms to Postgres traffic	Slow responses on every cache miss; pool exhaustion	training
postgres_down	docker kill the Postgres container	503 errors (database unavailable)	training
api_crash	docker kill the API container	Connection errors	training
redis_partition	Toxiproxy disables the Redis proxy (container healthy)	Looks exactly like redis_down	held out
redis*down*slow_db	Kill Redis and add 400 ms to Postgres	Errors, then slowness	held out

4.5 The six recovery actions (Ansible playbooks)

Action	What it does	Impact / cost	When it helps
none	Nothing (baseline)	none / 0.00	Never, but it measures the fault's natural behaviour
restart_redis	docker restart Redis, wait for /ready	medium / 0.05	Redis process crashed
restart_postgres	docker restart Postgres, wait for /ready	high / 0.20	Postgres process crashed
restart_api	docker restart the API, wait for /ready	medium / 0.10	API process crashed
cache_bypass	API serves reads from Postgres, skipping Redis	low / 0.05	Redis dead, slow or unreachable
warm_cache	Load all 1,000 items into Redis (TTL 10 min) with one query	low / 0.02	Slow database: hits avoid it

Why each fix works (or not) is the heart of the first result:

- **redis_latency:** restarting Redis cannot help: Redis is fine, the network to it is slow. Only bypassing the cache helps. This is exactly where a classic runbook fails.

- **postgres_latency:** bypassing the cache makes it worse (every read now hits the slow database). Warming the cache in one bulk query puts every item in Redis, so requests stop touching the slow database.
- **redis_partition (held out):** looks like a dead Redis, but the container is healthy, so a restart does nothing. Cache bypass works.
- **redis_down_slow_db (held out):** cache bypass is a trap (all load goes to the slow database). The right fix needs two coordinated steps: restart Redis, then warm the cache. No policy found this.

4.6 The safety validator

`controller/chaosctl/safety.py` checks every proposal, from every policy, in this order. The first rule that fails blocks the action and the reason is recorded:

1. **Approved environment only:** the Compose project must match `chaosos` or `chaosos-ci-*`.
2. **Allowlist:** the action must exist in `safety-policy.yml` and in the code. `drop_database` does not.
3. **No repeats:** an action may run once per incident (stops restart loops, a classic automation disaster).
4. **Budget:** at most 3 executed actions per incident.
5. **Approval for high impact:** `restart_postgres` is high impact. In experiments the isolated lab auto-approves it (logged); in on-call mode a human decides in the dashboard (no answer in 60 s = not run); `deny` mode always blocks it.

A second, independent layer: the Ansible playbooks refuse any service outside `allowed_services` (api, redis, postgres, frontend), so even a bug in the validator could not restart Grafana or Jenkins.

> **THE RED-TEAM TEST**
> The adversarial policy proposes `drop_database`, `restart_grafana`, repeated restarts and a high-impact action on purpose. In every run, the unsafe proposals were blocked with a clear reason and only allowlisted actions ran. Across all experiments, **zero unsafe actions executed**.

5. The responders: who decides what to do

A policy receives the symptoms plus what has already been tried in this incident, and proposes the next action. Every policy gets exactly the same inputs, and every proposal goes through the same safety validator. Code: `controller/chaosctl/policies.py`, `bandit.py`, `llm.py`, `agent.py`.

5.1 Rule-based runbook (the classic approach)

"Restart whatever looks broken, then restart the app." If the API is down, restart the API; if Redis looks broken, restart Redis; if Postgres looks broken, restart Postgres; finally restart the API. This is what many teams do, and it is our baseline.

5.2 Expert heuristic (hand-written rules)

Rules written after studying the training results: API down → restart API; Postgres unreachable → restart Postgres; Redis unreachable → cache bypass (then restart Redis); Postgres probe slower than 200 ms → warm cache; Redis probe slower than 200 ms → cache bypass. It is strong precisely because a human encoded the answers; that is also its weakness (someone must study every new failure).

5.3 LinUCB contextual bandit (machine learning)

Features (8): a bias term, API down, Redis unreachable, Postgres unreachable, Redis probe latency, Postgres probe latency, error rate, and p95 latency. Latencies are log-scaled into 0-1 (2 s and above saturate), so 500 ms and 2,000 ms are distinguishable but extreme values do not dominate.

Reward for each executed step: `max(0, 1 - time_to_heal / 20 s) - safety_cost(action)`. A step that heals the service immediately scores close to 1; one that never heals scores 0; risky actions pay a penalty (restart*postgres 0.20, restart*api 0.10, restart*redis and cache*bypass 0.05, warm_cache 0.02).

Model: for each action a, LinUCB keeps a matrix A*a (identity plus the sum of x·x^T) and a vector b*a (the sum of reward·x). The predicted reward is θ*a·x with ϑ*a = A*a*⁻¹·*b*a, and the uncertainty bonus is α·√(x^T A_a^{-1} x) with α = 0.5. At decision time the lab picks the highest prediction among actions not yet tried in the incident.

Training: offline, from logged steps of the **training failures only** (the code asserts that no held-out data leaks in). The learning curve replays the 179 logged steps in 30 random orders and scores how often the bandit picks the best measured action: it ends at 88%.

> **"ISN'T THE BANDIT JUST MEMORISING FAILURE NAMES?"**
> No. It never sees the name. It sees 8 symptom numbers. The proof is the held-out Redis partition: never trained on, it looks like "Redis unreachable", and the bandit chose cache bypass (3/3 recovered in 0.8 s) while the runbook restarted a healthy container and failed 0/3.

5.4 LLM policy (gpt-4o-mini)

The symptoms, the allowed actions with their descriptions, and what was already executed or blocked are sent as JSON. The system prompt is short:

```
You are an on-call SRE remediation agent for a web shop: an API that reads items
through a Redis cache backed by PostgreSQL. You receive live symptoms and must pick
exactly ONE next recovery action from the allowed list, or "give_up" if none can help.
Do not repeat actions already executed.
Reply with JSON only: {"action": "<name>", "reason": "<one sentence>"}
```

Temperature 0 (as repeatable as possible), JSON mode, and the model's reason is stored with each step. The API key lives only in the git-ignored `.env` and is never printed or logged.

5.5 LLM agent with tools (agentic AI)

The agent gets the same prompt and inputs **plus five tools**. It must call at least one investigation tool before it may decide ("look before you act"), and at most five before it is forced to decide:

Tool	What it returns	Safety
promql(query)	Any instant Prometheus query: error counters, cache hits/misses, dependency up, SLO ratios	Read-only
logs(service)	Last 2 minutes of a shop container's log, repeated lines collapsed	Only api, redis, postgres, frontend
containers()	Running / exited state and health of the shop containers	Read-only
readiness()	The API's /ready result: Redis/Postgres reachable, probe ms, cache bypass flag	Read-only
decide(hypothesis, action, reason)	Ends the investigation with one action or give_up	Still checked by the validator

Toxiproxy (the fault injector) is deliberately invisible to every tool, so the agent cannot cheat by reading the fault. Every tool call and its result is saved as an **investigation trail**, shown in the dashboard under each step.

5.6 The ablation variants

To learn *why* the LLM behaves as it does, we added one paragraph of **generic SRE guidance** (no failure-specific rules) and tested a 2 x 2 design: {no tools, tools} x {no guidance, guidance}:

```
SRE practice: a restart only helps when the process itself has crashed or hung; it does
not fix a slow or unreachable network, and it throws away warm state. Check whether a
component is down or merely slow before restarting it. Prefer the least disruptive
action
that restores user traffic, and think through each action's side effects on the other
components.
```

Variant	Tools	Guidance	Name in the code
LLM	no	no	llm
LLM + SRE guidance	no	yes	llm_sre
LLM agent + tools	yes	no	llm_agent
Agent + tools + guidance	yes	yes	llm*agent*sre

> **WHY AN ABLATION?**
> If we only compared "LLM" with "agent + guidance" and the second won, we could not say whether the tools or the guidance caused it. Changing one factor at a time isolates each effect. That is what makes it research rather than a demo.

5.7 Adversarial (red-team) policy

It exists only to attack the safety validator: it proposes `drop_database`, `restart_grafana`, repeated restarts and high-impact actions on purpose. It is never used to fix anything.

6. On-call mode: healing failures nobody announced

In experiments the controller knows what it broke. In real life nobody tells you. On-call mode makes the lab behave like production: steady user traffic, an SLO, burn-rate alerts, and an always-on agent that is woken by a page, the way a human on-call engineer is.

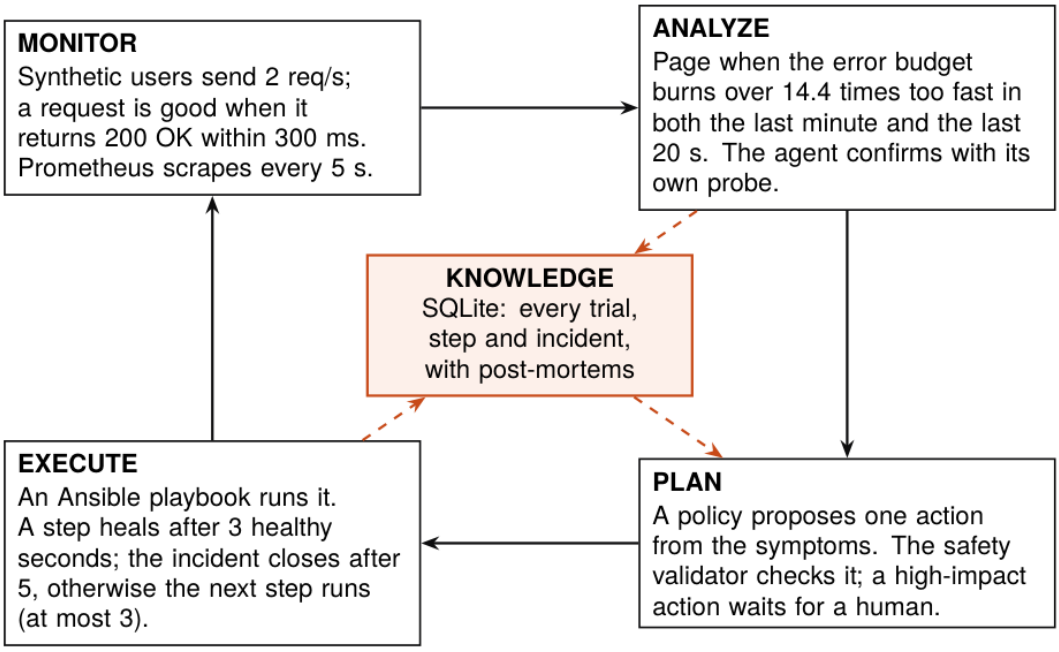


Figure 2. The on-call control loop mapped onto MAPE-K (Monitor, Analyze, Plan, Execute over shared Knowledge), the reference model for self-managing (autonomic) systems.

6.1 The story of one incident, step by step

1. **Fault.** Someone (the dashboard's Break-it button, the chaos monkey, or the CI drill) kills Redis. The agent is not told; the chaos log is written to a table the agent never reads.
2. **Users notice.** The synthetic users (2 requests/s) start counting errors.
3. **Prometheus notices.** Every 5 s it recomputes the bad-request ratio over 20 s and 1 min. When both exceed 14.4%, the alert `ShopSLOFastBurn` fires with severity page.
4. **Alertmanager pages the agent** through a webhook (`POST http://oncall:8095/alerts`).
5. **The agent checks the rules of engagement:** autopilot on? Is an experiment using the lab right now (the shared lab lock)? If so, it stands down.
6. **It confirms.** It sends its own probe traffic (10 req/s) for 4 s. If the probe is healthy, the page is stale (the problem already went away) and no incident is opened.
7. **It opens an incident** and runs the same remediation loop as experiments with the chosen responder (heuristic by default; bandit, LLM or LLM agent selectable in the dashboard).
8. **High-impact actions wait for you.** The dashboard shows an approval card (and the top bar turns orange on every page). Approve, deny, or let it expire: after 60 s it is not run (fail safe).
9. **It closes the incident only on sustained recovery** (5 healthy seconds), otherwise it **escalates** to a human and does not keep retrying (no loops). An escalated incident is closed automatically once the service is healthy again.

10. **It writes the post-mortem** with gpt-4o-mini from the incident record, and marks everything on Grafana.

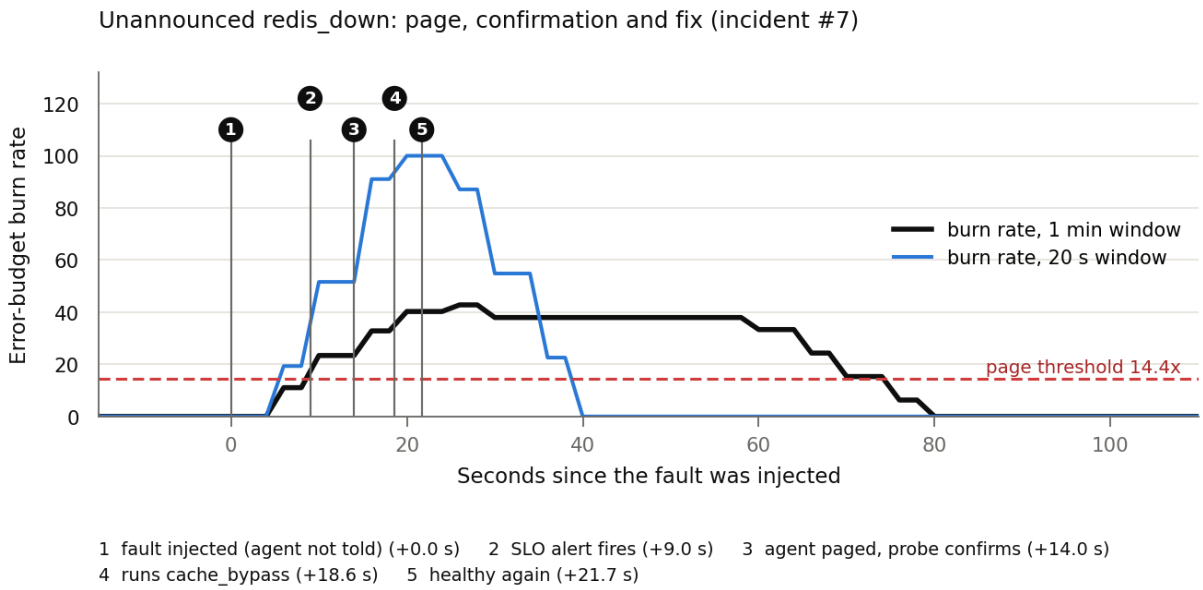


Figure 3. A real unannounced redis*down (incident #1): the 20 s burn rate jumps first; the page fires when the slower 1 min window also crosses 14.4 (+9 s); the agent confirms, runs cache*bypass and traffic is healthy at +21.7 s.

6.2 Why the agent re-checks every page

Alerts are based on windows of the past, so they lag: a page can still be firing for a few seconds after the problem is gone, and Alertmanager repeats a firing page every minute. Treating the page as a *trigger* and live traffic as the *truth* prevents the agent from "fixing" a healthy system. This is the same discipline human SREs follow: confirm before you act.

6.3 Human-in-the-loop approvals

- The safety policy says high-impact actions need approval in on-call mode (`approval.on_call: human`).
- The agent writes an approval request to the shared database; the dashboard shows it with the action, its description, the responder's reason and the live symptoms.
- Only the **first** decision counts (approve or deny); a second click is refused. After 60 s the request **expires** and the action is not run.
- Behind the HTTPS proxy, the approval is signed with the logged-in user's name (set by Caddy, not the browser).

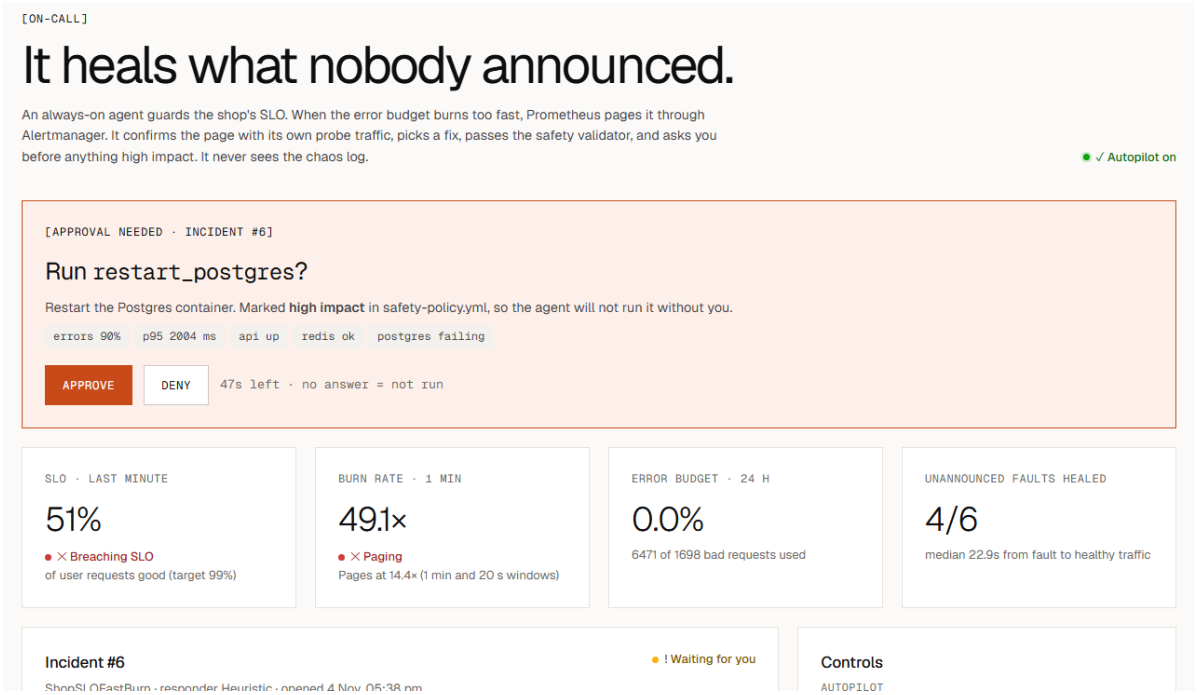


Figure 4. The On-call page during an unannounced postgres_down: the agent wants to restart Postgres (high impact) and waits for a human decision; the SLO is breaching and the burn rate is far above the 14.4x paging line.

6.4 The chaos side: Break it, chaos monkey, rollback

- **Break it:** inject a chosen or random failure from the dashboard, unannounced.
- **Chaos monkey:** when switched on, injects a random failure every ~10 minutes (randomised between 0.5x and 1.5x so it is not predictable), only when the lab is idle.
- **Rollback:** every injected fault is rolled back after 5 minutes at the latest, so an unattended lab never stays broken.
- **One fault at a time:** a new injection first rolls the previous one back.

6.5 The scorecard: how we prove it works

Every fault in the chaos log is matched to the first incident the agent opened within 10 minutes after it. Three times are measured from the moment of injection:

Metric	Definition	What it tests
Time to alert (MTTD)	Prometheus alert start - fault time	SLI + alert rule design
Time to engage	Incident opened (page received) - fault time	Alertmanager routing + webhook
Time to recover (MTTR)	Sustained healthy traffic - fault time	The whole loop, including the fix

6.6 Error budget on the dashboard

The On-call page shows the SLO over the last minute, the burn rate, and the 24-hour error budget: at 2 requests/s a day has 172,800 requests, so the budget is 1,728 bad requests. Every incident visibly spends some of it, which is how real teams decide between shipping features and fixing reliability.

> **IF A TEACHER ASKS WHY THE BUDGET SHOWS 0%**

> "On the day of our experiments we broke the shop on purpose more than 60 times in a few hours, so the budget is fully spent: that is the lab doing its job. In production, a spent error budget triggers an error-budget policy, for example freezing risky releases until reliability recovers; adding that rule to Jenkins is one of our future-work items."

7. The DevOps machinery

Everything in CHAOS OS is automated and written as code: the lab (Compose), the fixes (Ansible), the rules (YAML policies), the alerts (Prometheus rules with unit tests), the pipeline (Jenkinsfile and job XML), and the deployment (Ansible + Caddy).

7.1 The Jenkins pipeline, stage by stage

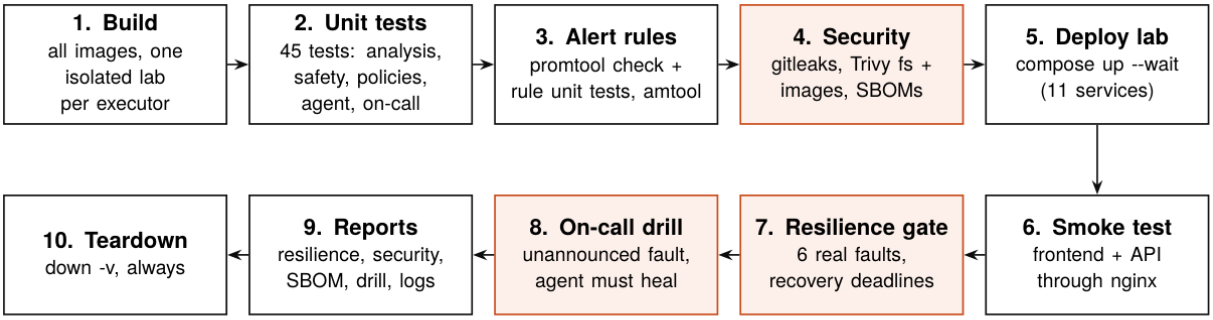


Figure 5. The Jenkins pipeline. Orange stages are gates that only chaos or security checks can fail; any failed stage stops the release.

Stage	What happens	Fails the build when
1. Build images	docker compose --profile tools build (all images)	Any image fails to build
2. Unit tests	45 Python tests inside the controller image	Any test fails
3. Alert rules	promtool check config, promtool test rules (4 cases), amtool check-config	A rule or route is wrong
4. Security scan	gitleaks (code + history), Trivy fs (deps + Dockerfiles), Trivy images, SBOMs	A secret, a HIGH/CRITICAL misconfiguration or fixable dependency CVE, or a fixable CRITICAL image CVE
5. Deploy lab	docker compose up -d --wait in an isolated project, no host ports	A service never becomes healthy
6. Smoke test	A throwaway container calls the shop through nginx	The shop does not answer correctly
7. Resilience gate	6 real faults with a fixed fix and a deadline (resilience-gate.yml)	Any recovery misses its deadline
8. On-call drill	Kill Redis without telling the agent; it has 120 s	No incident, or not healed in time
9. Reports	Archive resilience report, security reports, SBOMs, drill output, logs	-
10. Teardown	docker compose down -v (always, even on failure)	-

Isolation: each build uses the project name `chaosos-ci-<executor number>` and only the main Compose file (no host ports), so builds of different branches can run side by side and never touch the lab on your laptop. Sources reach scanner containers through stdin (`tar`), never bind mounts, because Jenkins talks to the host's Docker engine ("Docker outside of Docker").

Measured: build #6 of main finished **SUCCESS** in 5 min 46 s. Build images 36 s, Unit tests 3 s, Alert rules 3 s, Security scan 43 s, Deploy lab 21 s, Smoke test 1 s, Resilience gate 178 s, On-call drill 52 s.

7.2 The resilience gate and the demo bug

`resilience-gate.yml` lists the failures every release must survive and the deadline for each (roughly 2x the median measured locally, for headroom on a busier CI host):

Failure	Fix used	Deadline
redis_down	restart_redis	10 s
redis_down	cache_bypass	10 s
redis_latency	cache_bypass	10 s
postgres_latency	warm_cache	15 s
postgres_down	restart_postgres	15 s
api_crash	restart_api	20 s

> **THE DEMO THAT CONVINCES TEACHERS**
> Branch `demo/broken-cache-bypass` has a one-character bug: the cache-bypass endpoint writes the wrong flag key. It returns 200 OK, the Ansible playbook reports success, unit tests pass, the smoke test passes, but nothing actually changes. Only the resilience gate catches it: both cache-bypass checks never recover, so Jenkins shows that branch **red** while `main` stays **green**.

7.3 Alerts are code, and code is tested

`observability/prometheus/tests/slo_test.yml` feeds synthetic time series into the real rules and checks four behaviours: a real outage pages within seconds and stops paging after recovery; a 5-second blip does not page; slow responses burn the budget like errors; losing the synthetic traffic raises a ticket after 30 s. These run in CI with `promtool test rules`.

7.4 DevSecOps: the security scan

`scripts/security-scan.sh` runs four checks. Policy: any secret, any HIGH/CRITICAL misconfiguration or fixable dependency vulnerability, or a fixable CRITICAL vulnerability in an image fails the build. Accepted risks are listed in `.trivyignore.yaml`, each with a written justification.

Check	Tool	First run found	What we did
Secrets in code and history	gitleaks 8.21	0 leaks (10 commits + working tree)	Custom rule for current OpenAI key formats; .env and secrets never leave the machine
Python dependencies	Trivy 0.56 fs	ansible-core 2.17.5: 2 HIGH CVEs (CVE-2024-8775, CVE-2026-11332)	Upgraded to ansible-core 2.21.6 everywhere
npm dependencies (incl. dev)	Trivy fs	vite 5.4.10: 1 HIGH, 12 MEDIUM, 3 LOW; esbuild 1 MEDIUM; no lockfile	Vite 6.4.5 + plugin-react 4.7.0; package-lock.json + npm ci (reproducible)
Dockerfiles (IaC)	Trivy misconfig	12 Dockerfiles without a non-root USER (HIGH); 1 CRITICAL that was a false positive	Non-root for api, traffic, frontend (unprivileged nginx), Toxiproxy, Prometheus, Alertmanager, Grafana; 5 justified exceptions + 1 false positive documented
Built images	Trivy image	frontend base (nginx 1.27, old Alpine): 3 CRITICAL + 58 HIGH; Docker CLI and gosu binaries: 1 CRITICAL each (old Go)	nginx-unprivileged 1.30 + apk upgrade (0 left); Docker CLI 29.8; gosu CVE accepted until 2027-03-01 (unreachable code); CycloneDX SBOM per image (441 components)

> **JUSTIFIED EXCEPTIONS (ACCEPTED RISKS)**
> Controller, dashboard and the Ansible runner must use the host Docker socket to inject faults and restart containers, which requires root inside the container. They have no public ports in production and every action passes the safety validator. Postgres's official entrypoint drops root itself. One Trivy finding was a false positive (`ANSIBLE_HOST_KEY_CHECKING` contains the word KEY).

7.5 Deployment

- **Production overlay** (`deploy/docker-compose.prod.yml`): Caddy is the only public entry point, with automatic HTTPS (Let's Encrypt via a free sslip.io name), a bcrypt login, security headers, Grafana under /grafana, and restart policies on every service.
- **One-command provisioning** (`deploy/ansible/deploy.yml`, run by `deploy.ps1` from a container so it works on Windows): firewall rules (including Oracle Cloud's iptables quirk), Docker install, swap on small servers, git clone, secrets in an owner-only .env, seeding the experiment history, `compose up --wait`, and an HTTPS + login check from outside.
- **No server, no card:** `deploy/share.ps1` runs the same production overlay on the laptop with a Cloudflare quick tunnel; nothing is opened on the router, and the same login protects it.

7.6 The evidence dashboard, page by page

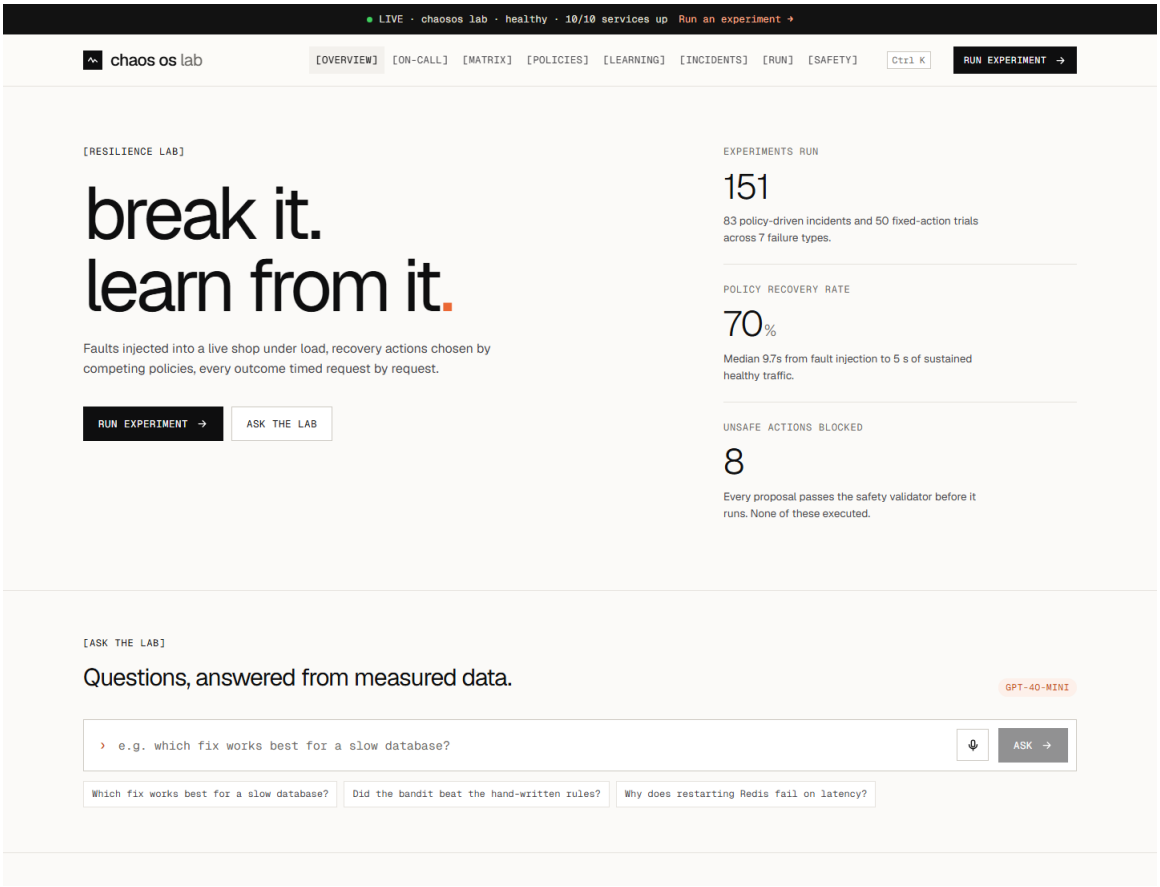


Figure 6. The evidence dashboard's Overview: headline numbers, Ask the lab (answers come from measured data only) and the leaderboard of responders per failure.

Page	What it shows / does
Overview	Headline numbers, Ask the lab (gpt-4o-mini answers only from measured data), AI analysis of results, charts
On-call	SLO, burn rate, 24 h error budget, live incident timeline, approvals, Break it, chaos monkey, scorecard, incident history with post-mortems
Matrix	Every failure x every fixed action: recovery time and failed requests
Policies	Ranked responders per failure, training and held out
Learning	The bandit's learning curve against the fixed rules
Incidents	Every experiment as a timeline: symptoms, actions, verdicts, the LLM's reason, the agent's tool trail, post-mortem
Run	Launch an experiment (failure + responder or fixed action) and watch the live log
Safety	The safety rules, blocked proposals, and a one-click red-team test

7.7 Generative AI features (gpt-4o-mini only)

- **Decide:** the `llm` policies pick actions; the `llm_agent` policies investigate with tools first.
- **Explain:** post-mortems for experiments and on-call incidents, and a results analysis.

- **Ask the lab:** free questions answered from the measured data only (the model is told never to invent numbers, and to say when the data cannot answer).
- **Cost and privacy:** temperature 0, small prompts; the whole project's AI usage costs cents. The key is only in `.env` (git-ignored), never in code, logs or chat. CI runs without a key.

8. Hands-on: running and reproducing everything

Every command below was run while building the project. Use PowerShell or Git Bash from the project folder, with Docker Desktop running.

8.1 Start the lab

```
docker compose up -d --build --wait      # the lab + dashboard + on-call agent (first
build: minutes)
python scripts/smoke_test.py             # should print PASS
```

Open	Address
Evidence dashboard	http://localhost:8090
Shop	http://localhost:8080
Grafana (Service Health)	http://localhost:3000 (admin / admin)
Prometheus	http://localhost:9090
Alertmanager	http://localhost:9093
API docs	http://localhost:8000/docs
Jenkins	http://localhost:8081 (start it with: docker compose -f jenkins/docker-compose.yml up -d)

> **NEVER RUN `DOCKER COMPOSE DOWN -V` UNLESS YOU MEAN IT**
> `-v` deletes the volumes, including the experiment history (every trial, incident and the trained bandit). Plain `docker compose down` is safe.

8.2 Run experiments from the command line

```
# One failure, one responder
docker compose run --rm controller run --scenario redis_latency --policy bandit
# The fixed-action matrix (training data for the bandit)
docker compose run --rm controller run --scenario all --action
restart_redis,restart_postgres,restart_api,cache_bypass,warm_cache --reps 2 --timeout
30
# Train the bandit and see its learning curve
docker compose run --rm controller train
docker compose run --rm controller learning-curve
# Compare responders, then the held-out generalisation test
docker compose run --rm controller run --scenario all --policy
rule_based,heuristic,bandit,llm --reps 2 --timeout 90
docker compose run --rm controller run --scenario held_out --policy
rule_based,heuristic,bandit,llm --reps 3 --timeout 90
# The LLM ablation (2 x 2)
docker compose run --rm controller run --scenario all --policy
llm,llm_sre,llm_agent,llm_agent_sre --reps 2 --timeout 90
# Red-team the safety validator
docker compose run --rm controller run --scenario redis_down --policy adversarial
# Summaries and AI write-ups
docker compose run --rm controller report --by policy
docker compose run --rm controller explain
```

8.3 On-call mode

```
# An unannounced drill: passes only if the agent heals it within 120 s
docker compose run --rm controller drill --scenario redis_down --within 120
```

```
# The full measurement campaign (all 7 failures, 3 times each, operator approves after
5 s)
python scripts/oncall_campaign.py --reps 3 --policy heuristic --out oncall-
campaign.json
# Watch the agent think
docker logs -f chaosos-oncall-1
```

In the dashboard: **On-call** page → choose a failure → **Break it**. Approve or deny when the orange approval card appears.

8.4 CI, alert-rule tests and the security scan

```
docker compose run --rm --no-deps --entrypoint promtool prometheus test rules
/etc/prometheus/tests/slo_test.yml
docker compose run --rm --entrypoint python controller -m unittest discover -s tests -v
sh scripts/security-scan.sh              # reports and SBOMs in security-reports/
```

Jenkins builds every branch automatically within 5 minutes of a push (it polls GitHub). Open the **chaos-os-ci** job to see the stage view and the archived reports.

8.5 Regenerate the paper's figures and this guide

```
python docs/paper/make_figures.py --campaign oncall-campaign.json --drill-incident <id>
cd docs/paper && tectonic -X compile chaos-os-ieee.tex
cd docs/study-guide && node build.js
```

8.6 Share it online

```
.\deploy\share.ps1           # public HTTPS link from your laptop (Cloudflare tunnel),
with login
.\deploy\share.ps1 -Stop     # back to the normal local lab
.\deploy\deploy.ps1 -Server <ip>   # full deployment to a cloud VM
```

8.7 Troubleshooting

Symptom	Fix
"the lab is busy" when starting a run	An experiment or an on-call incident owns the lab; wait (at most ~2 min)
The top bar says fault active	Dashboard → Overview → Reset lab
Docker commands fail with "cannot connect"	Start Docker Desktop and wait for "Engine running"
LLM policies show (unavailable)	Put OPENAI*API*KEY in .env, then docker compose up -d
Bandit shows (unavailable)	Train it: docker compose run --rm controller train
No page after Break it	Check Prometheus targets (traffic must be up) and Alertmanager at :9093
Port 8080 already in use	Another program uses it; stop it or change the port in docker-compose.override.yml

9. Results

All numbers come from the experiment database (`results.json` is generated from it by `docs/paper/make_figures.py`). Load is 20 requests/s, so a 60 s outage is about 1,200 failed requests. Format in the tables: recovered/trials · median recovery time (min–max).

9.1 Result 1: every failure needs a different fix

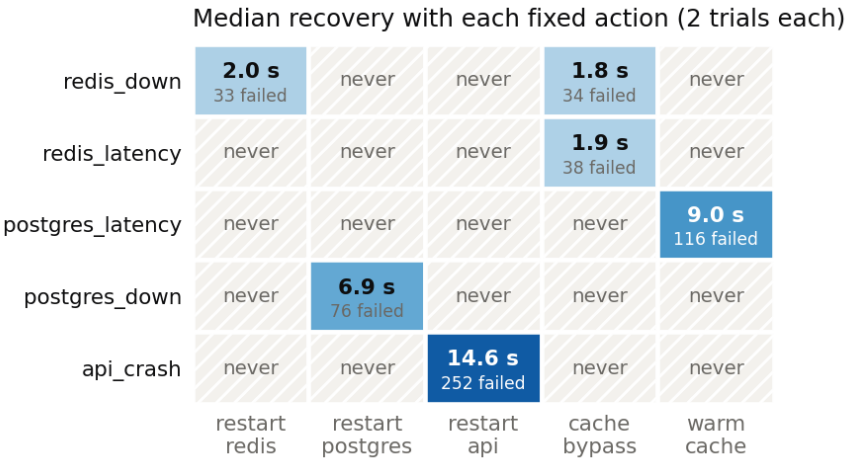


Figure 7. Fixed-action matrix: median recovery time and failed requests for every failure and every action (2 trials each). Hatched cells never recovered.

Failure	restart_redis	restart_postgres	restart_api	cache_bypass	warm_cache
redis_down	2/2 · 2.0 s (1.9–2.0)	✕ 0/2	✕ 0/2	2/2 · 1.8 s (1.8–1.9)	✕ 0/2
redis_latency	✕ 0/2	✕ 0/2	✕ 0/2	2/2 · 1.9 s (1.9–2.0)	✕ 0/2
postgres_latency	✕ 0/2	✕ 0/2	✕ 0/2	✕ 0/2	2/2 · 9.0 s (8.9–9.2)
postgres_down	✕ 0/2	2/2 · 6.9 s (5.8–7.9)	✕ 0/2	✕ 0/2	✕ 0/2
api_crash	✕ 0/2	✕ 0/2	2/2 · 14.6 s (14.5–14.7)	✕ 0/2	✕ 0/2

Each row has exactly one action that recovers quickly, and it is a different action in every row. The wrong choice does not just fail; it keeps the outage going until the timeout, costing hundreds to more than a thousand failed requests.

9.2 Result 2: policies on training and held-out failures

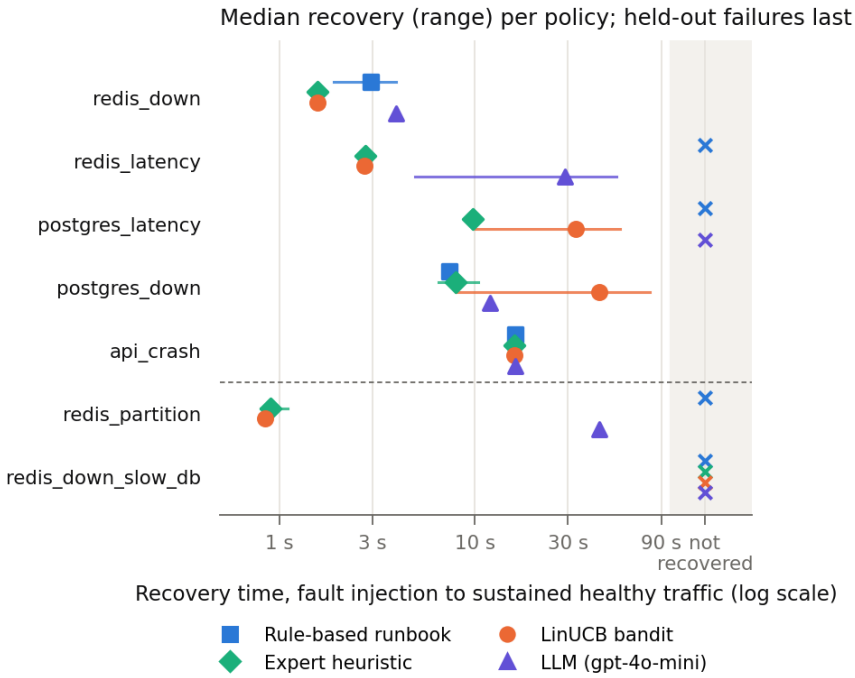


Figure 8. Median recovery (range) per responder. Held-out failures (below the dashed line) were never used for design or training. X = not recovered.

Failure	Rule-based	Heuristic	Bandit	LLM
redis_down	2/2 · 2.9 s (1.9–4.0)	2/2 · 1.6 s	2/2 · 1.6 s	2/2 · 4.0 s
redis_latency	✕ 0/2	3/3 · 2.8 s (2.8–2.9)	2/2 · 2.7 s (2.7–2.8)	2/2 · 28.9 s (5.0–52.9)
postgres_latency	✕ 0/2	2/2 · 9.8 s (9.7–9.8)	2/2 · 32.7 s (10.1–55.3)	✕ 0/2
postgres_down	2/2 · 7.4 s (7.3–7.4)	3/3 · 8.0 s (6.5–10.4)	2/2 · 43.3 s (8.0–78.5)	2/2 · 12.0 s
api_crash	2/2 · 16.1 s (16.0–16.2)	2/2 · 16.0 s (15.9–16.0)	2/2 · 16.0 s	2/2 · 16.1 s (16.1–16.2)
redis_partition (held out)	✕ 0/3	3/3 · 0.9 s (0.8–1.1)	3/3 · 0.8 s	3/3 · 43.4 s (43.4–43.5)
redis*down*slow_db (held out)	✕ 0/3	✕ 0/3	✕ 0/3	✕ 0/3

Totals over all seven failures: rule-based 6/16, heuristic 15/18, bandit 13/16, LLM 11/16 recovered. The runbook fails exactly where "restart it" is the wrong answer. The bandit matches the expert rules and generalises to the Redis partition. Nobody recovers from the compound failure, which needs two coordinated steps.

9.3 Result 3: the bandit learns from experience

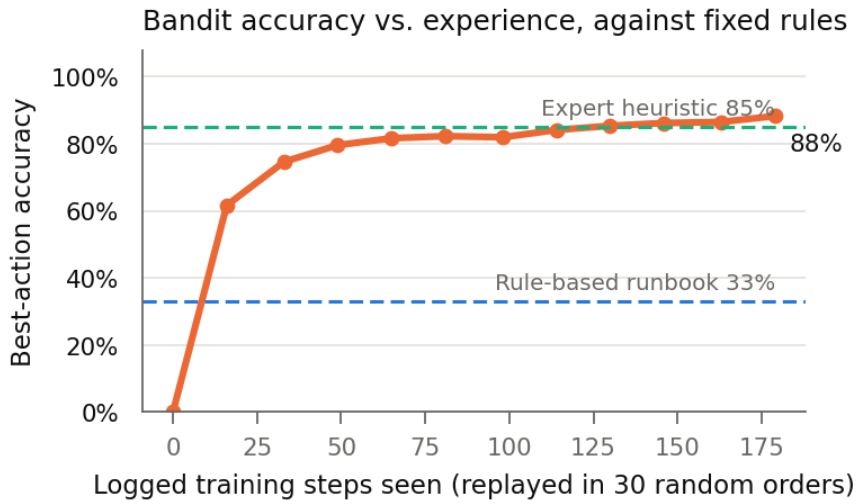


Figure 9. Bandit best-action accuracy as it sees more logged steps (30 random replay orders), against the fixed rules.

Replayed over all 179 logged training steps, the bandit reaches 88% best-action accuracy; the expert heuristic scores 85% and the rule-based runbook 33% on the same contexts. (An earlier snapshot with only the 52 fixed-action steps reached 98%; the pool now also contains every logged step from the policy and LLM runs, including their mistakes.)

9.4 Result 4: the LLM ablation (tools vs. advice)

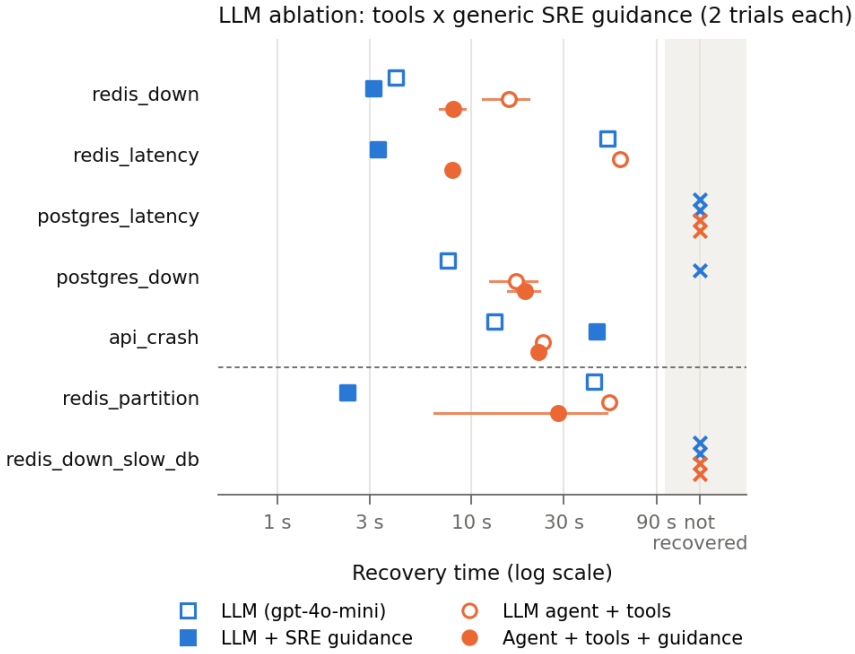


Figure 10. LLM ablation: blue = no tools, orange = tools; hollow = no guidance, filled = generic SRE guidance. Median recovery (range), 2 trials each.

Failure	LLM	LLM + guidance	Agent + tools	Agent + tools + guidance
redis_down	2/2 · 4.1 s (4.0–4.3)	2/2 · 3.1 s (3.0–3.3)	2/2 · 15.6 s (11.4–19.8)	2/2 · 8.1 s (6.9–9.3)
redis_latency	2/2 · 50.1 s (50.0–50.3)	2/2 · 3.3 s (3.1–3.5)	2/2 · 58.6 s (58.5–58.7)	2/2 · 8.0 s (7.7–8.3)
postgres_latency	✕ 0/2	✕ 0/2	✕ 0/2	✕ 0/2
postgres_down	2/2 · 7.6 s (7.2–8.0)	✕ 0/2	2/2 · 17.1 s (12.5–21.6)	2/2 · 18.9 s (15.4–22.4)
api_crash	2/2 · 13.2 s (12.8–13.5)	2/2 · 44.1 s (42.6–45.5)	2/2 · 23.5 s (23.3–23.7)	2/2 · 22.1 s (21.0–23.3)
redis_partition (held out)	2/2 · 43.0 s (42.5–43.5)	2/2 · 2.3 s	2/2 · 51.1 s (50.9–51.3)	2/2 · 28.0 s (6.5–49.5)
redis*down*slow_db (held out)	✕ 0/2	✕ 0/2	✕ 0/2	✕ 0/2

Totals: LLM 10/14, LLM + guidance 8/14, agent + tools 10/14, agent + tools + guidance 10/14 recovered.

Advice changes the habit, not the reasoning. Without guidance, both LLM variants restarted Redis first on every Redis fault, which cost 43-59 s on the two faults a restart cannot fix (redis*latency, redis*partition). With the SRE paragraph and no tools, the model chose cache bypass first in **all 14** of its trials: the fastest LLM result where bypass is right (redis_latency 50.1 s -> 3.3 s, partition 43.0 s -> 2.3 s), but harmful where a restart is right: it never restarted the crashed database (0/2, against 2/2 in 7.6 s without advice) and was 3x slower on the crashed API (44.1 s against 13.2 s).

Tools alone only cost time. The agent made 3.6 tool calls per decision on average (1 to 5, over 45 decisions). Without guidance it picked the same first action as the plain LLM on every fault except the slow database (where neither helped), so the investigation added 8-12 s per incident (e.g. redis_down 4.1 s -> 15.6 s) without improving any outcome.

Tools + guidance is the most robust. It recovered every fault that any LLM variant recovered (10/14): cache bypass for Redis faults (8.0-8.1 s), and a restart for the crashed database and API (18.9 s, 22.1 s). On postgres_down it checked readiness, metrics and logs, wrote the hypothesis "PostgreSQL is down", and restarted it. The guided LLM without tools saw the same "Postgres unreachable" symptom but bypassed the cache, so the investigation step made the difference (more evidence, an explicit hypothesis, or both; two trials cannot separate them). One of its two partition trials still restarted first (6.5 s and 49.5 s): LLMs vary even at temperature 0.

What no LLM found: none used warm_cache for the slow database (0/8) or the two-step fix for the compound failure (0/8). The bandit and the expert heuristic fix the slow database reliably because they learned from measured outcomes. Lesson to say in the viva: LLMs were best as explainers and as cautious first responders behind the safety shield, not as optimisers of recovery time, and prompt text must be tested like code.

9.5 Result 5: on-call mode heals unannounced failures

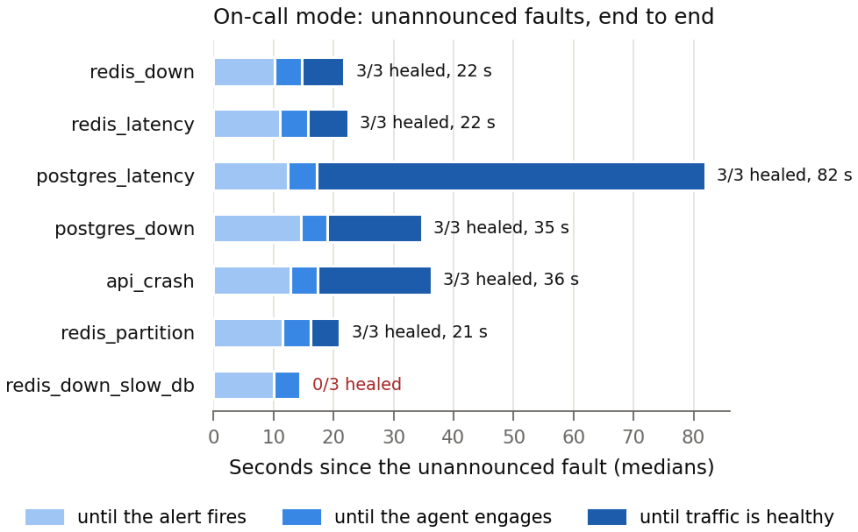


Figure 11. On-call campaign: medians of time to alert, to engage and to healthy traffic, measured from the unannounced fault.

Failure	Healed	Alert	Agent engaged	Healthy	Actions taken
redis_down	3/3	10.2 s	14.7 s	21.7 s	cache_bypass
redis_latency	3/3	11.1 s	15.8 s	22.4 s	cache_bypass
postgres_latency	3/3	12.3 s	17.2 s	81.9 s	restart*postgres,warm*cache; warm_cache
postgres_down	3/3	14.5 s	19.0 s	34.7 s	restart_postgres
api_crash	3/3	12.8 s	17.4 s	36.3 s	restart_api
redis_partition	3/3	11.5 s	16.1 s	21.0 s	cache_bypass
redis*down*slow_db	0/3	10.0 s	14.4 s	–	cache*bypass,restart*postgres; cache*bypass,restart*postgres,restart_redis

Over all faults in the chaos log: 23/27 healed by the agent; median time to alert 11.8 s, median time to healthy traffic 23.9 s; 4 incidents escalated to a human. Approvals: 10 approved, 1 denied.

Detection is steady, recovery depends on the fix. The alert fired 10-15 s after every kind of fault (it depends on the burn-rate windows, not on the fault) and the agent engaged about 5 s later. Redis faults healed by cache bypass were healthy after about 21-22 s; crashed services that needed a restart after 35-36 s (including the 5 s human approval for restart_postgres).

The slow database was hardest (median 81.9 s): connection-pool timeouts made it look unreachable, so the agent sometimes asked to restart Postgres first and only then warmed the cache: the same symptom ambiguity the bandit showed in experiments.

Escalation worked as designed: all 3 compound faults were escalated after the 3-action budget, with no retry loops, and each escalation closed by itself once the lab was healthy again. 8 high-impact actions ran, every one after an approval.

9.6 Result 6: safety, CI and security

- **Safety:** zero unsafe actions executed across all experiments and on-call incidents. Red-team proposals (`drop_database`, `restart_grafana`, repeats) were always blocked with a recorded reason, and the validator also stopped LLM agents from repeating `restart_postgres`.
- **Resilience gate:** the one-character bug on `demo/broken-cache-bypass` passes unit and smoke tests but fails the gate; `main` passes all six checks.
- **Security:** no secrets in 10 commits; 2 HIGH CVEs in ansible-core and 1 HIGH + 12 MEDIUM in the Vite build tool upgraded; frontend base image 61 HIGH/CRITICAL -> 0 (nginx-unprivileged 1.30 + apk upgrade); Docker CLI 27.3 -> 29.8 (1 CRITICAL fixed); 7 containers made non-root; the scan now passes with documented exceptions (5 containers that must run as root, 1 false positive, 1 CVE accepted until 2027-03-01).
- **CI:** main #6 passed every stage in 5 min 46 s (gate recoveries 2.0-12.2 s against 10-20 s deadlines; the unannounced drill was healed 19.6 s after the fault); the demo branch failed exactly the two cache-bypass gate checks.

10. Defending the project

Teachers reward teams that know the weak spots of their own work. Say the limitations before you are asked, and connect each one to a concrete next step.

10.1 Limitations (say these yourself)

- **One machine, one lab.** Docker Desktop on a laptop; absolute times depend on hardware and include probing and Ansible overhead. The comparisons between responders are fair because they share conditions.
- **Small samples.** 2-3 trials per (failure, responder) pair. We show ranges, not confidence intervals.
- **Simple workload.** One read endpoint with random items; real traffic has hot keys, writes and bursts.
- **Seven fault types.** No disk, CPU, memory or multi-service cascading faults yet.
- **One-step decisions.** Policies pick one action at a time; the compound failure needs a coordinated two-step plan, and nothing solved it.
- **The heuristic was written after seeing the data**, which is why it is strong.
- **LLM results vary** run to run even at temperature 0, and depend on one vendor model (gpt-4o-mini).
- **On-call detection is about 10-20 s** because the SLI uses 2 requests/s and 5 s scrapes; production traffic would give faster, finer signals.
- **Some containers run as root** because they control Docker; this is documented and isolated, not ideal.

10.2 Future work

- Sequence-aware policies (planning or reinforcement learning behind the same safety shield) for compound failures.
- A Kubernetes port with Chaos Mesh as the fault injector and canary releases gated on the SLO.
- More trials with confidence intervals and significance tests.
- Safe online exploration for the bandit in CI.
- Distributed tracing (OpenTelemetry) as an extra tool for the LLM agent.
- Error-budget policy: freeze releases automatically when the budget is spent.

10.3 The live demo (about 7 minutes)

10 minutes before: start Docker Desktop, run `docker compose up -d --wait` and `docker compose -f jenkins/docker-compose.yml up -d`, open tabs for the dashboard (:8090), Grafana (:3000), Jenkins (:8081) and GitHub, and do one warm-up Break-it so the first live run is not a cold start. Check that the top bar says healthy.

Time	Do	Say
0:00	Dashboard → Overview	"Everything you see is measured: trials, recovery, blocked unsafe actions."
0:40	On-call → Failure: Surprise me → Break it	"I just broke the shop. The agent was not told what or when."
1:00	Watch the incident timeline appear	"Users noticed, the SLO burned 14x too fast, Alertmanager paged the agent, it confirmed with its own traffic, chose a fix, the validator allowed it."
2:00	Open the incident → post-mortem	"gpt-4o-mini wrote this from the incident record only."
2:45	Break it: postgres_down → approval card → Approve	"Restarting the database is high impact, so the agent asks me. No answer in 60 s means it does nothing."
3:45	Incidents → an LLM agent trial → tool trail	"The agent investigated with real tools before deciding; every call is auditable."
4:30	Grafana → SLO and on-call row	"Here is the burn rate crossing the 14.4 line, and our annotations."
5:15	Jenkins → main (green) vs demo/broken-cache-bypass (red)	"A one-character bug passed unit and smoke tests. The resilience gate blocked it. Every build also scans for secrets and CVEs and runs an unannounced on-call drill."
6:15	Overview → Ask the lab (Ctrl+K)	Let a teacher ask something; the answer comes from our data.

IF SOMETHING GOES WRONG

No internet: skip the AI parts; everything else is local. A run looks stuck: Overview → Reset lab. "Lab is busy": an experiment or incident is running; wait. Jenkins down: show screenshots from this guide.

10.4 Viva questions with model answers

The project in general

WHAT IS CHAOS OS IN ONE SENTENCE?

A lab that injects real failures into a web service, lets different responders fix them behind a safety validator, measures every recovery, learns the best fix, heals unannounced failures with an on-call agent, and gates releases on the results.

WHY IS THIS A DEVOPS PROJECT AND NOT JUST AN AI PROJECT?

The AI is one responder. The project is a complete DevOps loop: infrastructure as code (Compose, Ansible), CI/CD with Jenkins and four kinds of gates, monitoring and alerting (Prometheus, Grafana, Alertmanager), SLOs and incident response, DevSecOps scanning, and automated HTTPS deployment.

WHY BUILD YOUR OWN LAB INSTEAD OF USING CHAOS MESH, LITMUSCHAOS OR GREMLIN?

Those tools inject faults. We needed to compare recovery strategies, learn from them and gate releases on the result. A Kubernetes port with Chaos Mesh as the injector is future work; our policies, safety layer and gates would stay the same.

WHY DOCKER COMPOSE AND NOT KUBERNETES?

It runs on a student laptop, starts in seconds and keeps every fault easy to understand. The ideas (SLOs, safety shield, gates) do not depend on the orchestrator.

Measurement

HOW EXACTLY DO YOU DEFINE "RECOVERED"?

Five consecutive 1-second windows in which at least 99% of requests succeed and p95 latency is at most 300 ms, measured per request by our own load generator. Recovery time is the send time of the first request after the last bad one.

WHY NOT MEASURE RECOVERY WITH PROMETHEUS?

It scrapes every 5 s, too coarse for 1-2 s recoveries. The load generator times every request. Prometheus is used where minute-scale signals are right: SLO alerts.

WHY IS THE LOAD GENERATOR "OPEN MODEL"?

It sends a request every 50 ms regardless of replies. A closed model would slow down during an outage and hide the damage.

ARE 2-3 TRIALS ENOUGH?

Enough to see clear differences such as 0/2 vs 2/2 and 50 s vs 3 s, not for confidence intervals. We report ranges and list this as a limitation.

WHY ARE SOME TRIALS MARKED INVALID?

If the 10 s before injection were not healthy, the trial did not start from steady state, so its result would be meaningless; it is excluded automatically.

Failures and fixes

WHY DOES RESTARTING REDIS NOT FIX REDIS_LATENCY?

Redis is healthy; the network to it is slow (800 ms, above the API's 500 ms timeout). Restarting a healthy process changes nothing. Cache bypass removes Redis from the request path.

WHY DOES CACHE BYPASS MAKE A SLOW DATABASE WORSE?

Every read then goes to the slow database, so the connection pool fills up. Warming the cache in one bulk query keeps reads off the database instead.

WHAT IS THE COMPOUND FAILURE AND WHY IS IT UNSOLVED?

Redis killed while Postgres is slow. Cache bypass is a trap (all load on the slow DB); the fix is restart Redis then warm the cache. Our policies choose one action at a time from single-fault experience and never found that sequence.

WHY IS RESTART_POSTGRES HIGH IMPACT?

It drops every database connection and in-flight query. It is the most disruptive allowed action, so it costs the most in the reward and needs approval in on-call mode.

Policies and machine learning

WHAT IS A CONTEXTUAL BANDIT AND WHY NOT DEEP REINFORCEMENT LEARNING?

It learns which action is best for a given context from rewards. Our decisions are judged by their immediate effect, data is small (hundreds of steps), and we want explainability. A linear bandit trains in milliseconds and its weights can be read.

WHAT DOES THE BANDIT SEE AND HOW IS IT REWARDED?

8 features: bias, API down, Redis unreachable, Postgres unreachable, log-scaled Redis and Postgres probe latency, error rate, p95. Reward = max(0, 1 - time*to*heal/20 s) minus the action's safety cost.

ISN'T IT MEMORISING THE FAILURE NAMES?

It never sees them. The held-out Redis partition proves it: never trained on, the bandit still chose cache bypass (3/3, 0.8 s) while the runbook failed 0/3.

HOW DO YOU STOP TEST DATA LEAKING INTO TRAINING?

Training reads only steps from training failures, and the code asserts that no held-out scenario is present. Held-out failures were run only after policies were frozen.

WHY IS THE HEURISTIC AS GOOD AS THE BANDIT?

A person wrote it after studying the training results, so it encodes the answers. The bandit reaches the same level without anyone writing rules, which scales to many more failures than a person can study.

WHY DID THE LEARNING-CURVE NUMBER CHANGE FROM 98%?

98% was a snapshot on the 52 fixed-action steps. The curve now replays every logged step, including the policies' and LLMs' mistakes, and the best action per failure is re-measured from all of them. The conclusion (bandit far above the runbook, close to the expert rules) is unchanged.

LLM and agent

WHY GPT-4O-MINI?

It is cheap and fast enough to sit inside an incident loop, and using one model keeps comparisons fair. The whole project's AI usage cost a few cents.

HOW DO YOU STOP THE LLM FROM DOING SOMETHING DANGEROUS?

It only proposes. Every proposal passes the safety validator (environment, allowlist, no repeats, budget, approval), and Ansible enforces its own allowlist. In our runs the validator blocked the agent from repeating restart_postgres.

WHAT CAN THE LLM AGENT DO, AND CAN IT CHEAT?

Read-only tools: PromQL queries, recent container logs, container states, the readiness check. Toxiproxy (the fault injector) is invisible to every tool, so it must diagnose from symptoms like a human.

WHAT DID THE ABLATION SHOW?

Advice changed the habit, not the reasoning: with the SRE paragraph the model chose cache bypass first in all 14 trials, so Redis latency fell from 50 s to 3.3 s but the crashed database was never restarted. Tools alone added 8-12 s without changing choices. Tools plus advice handled both kinds of fault (10/14, the best LLM result). No LLM found warm_cache for the slow database; the bandit did, from measured data.

ISN'T THE GUIDANCE JUST GIVING IT THE ANSWERS?

No failure or action names are in it: only general SRE principles found in any SRE book (restarts don't fix slow networks; check down vs. slow). The same paragraph was given with and without tools, so the comparison is fair.

ARE LLM RESULTS REPRODUCIBLE?
Not perfectly, even at temperature 0, which is why we ran each pair twice and report ranges, and keep every reason and tool call in the database for audit.

WHERE IS THE OPENAI KEY? COULD IT LEAK?
Only in the git-ignored .env. It is never printed or logged; errors strip the request; gitleaks scans every commit and the history in CI; CI itself runs without a key.

On-call mode

WHAT IS THE DIFFERENCE BETWEEN AN EXPERIMENT AND ON-CALL MODE?
In an experiment the controller knows the fault and the start time. In on-call mode the agent is woken only by an SLO alert and must work out what is wrong from symptoms.

EXPLAIN SLO, ERROR BUDGET AND BURN RATE.
SLO: 99% of user requests good. Error budget: the 1% that may be bad. Burn rate: how fast it is being spent; 14.4x is the standard paging threshold from the SRE Workbook (2% of a 30-day budget in one hour).

WHY TWO ALERT WINDOWS?
The long window (1 min) proves it is a real problem, not a blip; the short window (20 s) proves it is still happening, so the page stops soon after recovery. Our rule tests prove both behaviours.

WHY DOES DETECTION TAKE 10-20 SECONDS?
Five-second scrapes, rate windows and 2 requests/s of synthetic traffic. Faster would mean more false pages; it is the classic speed vs. precision trade-off.

WHY DOES THE AGENT RE-CHECK AFTER A PAGE?
Alerts lag behind reality. The page is a trigger; live traffic is the truth. This prevents "fixing" a system that is already healthy.

WHAT IF NOBODY APPROVES A HIGH-IMPACT ACTION?
After 60 s it expires and is not run (fail safe). The agent tries other safe options or escalates to a human.

WHAT IF THE AGENT CANNOT FIX IT?
It escalates and stops acting (no loops). The incident closes automatically when the service is healthy again, and faults are rolled back after 5 minutes.

HOW DO YOU KNOW THE AGENT DID NOT CHEAT?
It runs in its own container and never reads the chaos log; its only inputs are alerts, metrics, logs and its own probe traffic. The scorecard compares its incidents with the chaos log afterwards.

WHAT STOPS AN EXPERIMENT AND THE AGENT FROM FIGHTING OVER THE LAB?
A shared lab lock on the results volume. Only one actor (an experiment or an incident) owns the lab at a time; the agent stands down while an experiment runs.

DevOps, CI and security

WALK ME THROUGH THE PIPELINE.
Build, unit tests, alert-rule tests, security scan, deploy an isolated lab, smoke test, resilience gate, on-call drill, archive reports, teardown. Any failure stops the release.

WHAT DOES THE RESILIENCE GATE CATCH THAT TESTS MISS?

Behaviour under failure. The demo branch's one-character bug returns 200 OK and passes unit and smoke tests, but cache bypass silently does nothing; the gate's recovery deadlines fail the build.

HOW DO BUILDS AVOID INTERFERING WITH EACH OTHER?

Each build runs its own Compose project (chaosos-ci-<executor>) without host ports and tears it down with -v at the end.

WHAT DID THE SECURITY SCAN FIND?

No secrets. Two HIGH CVEs in ansible-core (upgraded), one HIGH and twelve MEDIUM advisories in the Vite build tool (upgraded), no lockfile (added, npm ci), twelve Dockerfiles running as root (seven fixed, five justified, one false positive), and fixable CRITICAL CVEs in three images: an old Alpine base under nginx (61 findings, now 0), an old Docker CLI (upgraded), and gosu in the official Postgres image (accepted until March 2027 because it handles no untrusted input). Remaining HIGH findings are reported, not hidden.

WHY DO SOME CONTAINERS STILL RUN AS ROOT?

They control the host's Docker engine to inject faults and restart containers, which needs root in the container. They have no public ports and every action is validated; the exceptions are written down with reasons.

WHAT IS AN SBOM AND WHY ARCHIVE IT?

A list of every package in an image. When a new vulnerability is announced, you can tell instantly which builds contain the affected package.

HOW IS IT DEPLOYED AND SECURED?

Caddy is the only public entry point: automatic HTTPS, a bcrypt login, security headers; everything else is private on Docker's network. Ansible provisions a server in one command; without a server, a Cloudflare tunnel shares the laptop behind the same login.

HOW DO YOU KNOW THE ALERT RULES ARE RIGHT?

promtool unit tests replay synthetic series through the real rules: an outage pages, a blip does not, slowness counts as bad, and losing traffic raises a ticket.

Wrap-up

WHAT ARE THE MAIN LIMITATIONS?

One laptop, small samples, a simple workload, seven fault types, one-step decisions (the compound failure is unsolved), and LLM variability.

WHAT WOULD YOU DO NEXT?

Sequence-aware policies behind the same safety shield, a Kubernetes port with Chaos Mesh and SLO-gated canaries, more trials with confidence intervals, and safe online learning in CI.

10.5 Who explains what (suggested split for four people)

Member	Owns	Chapters
Member 1	The problem, architecture, how experiments and recovery measurement work, the failures	1, 3, 4
Member 2	Responders: runbook, heuristic, bandit, LLM, agent; learning curve; the ablation	5, 9.1-9.4
Member 3	On-call mode: SLOs, burn-rate alerts, the agent, approvals, safety validator, scorecard	2.4-2.5, 4.6, 6, 9.5
Member 4	DevOps: Jenkins, resilience gate, security scan, deployment, dashboard; runs the live demo	7, 8, 10.3

FINAL TIP

When you are unsure, go back to the evidence: "we measured it; here is the number; here is the trial in the dashboard". That is the whole spirit of the project.

11. Code tour: where each idea lives

When a teacher says "show me the code", open the file below and explain the function in two sentences. All paths are relative to the repository root.

11.1 The experiment engine (controller/chaosctl)

Idea	File → function	What to say
Open-model load	load.py → LoadGenerator.run	Sends on a fixed schedule whatever the replies; records every request
Recovery definition	analysis.py → analyze, healthy_streak	1 s windows, 99% / 300 ms, 5 healthy windows, sub-window recovery time
One trial	experiment.py → run_trial	Reset, warm up, inject, detect, remediate, measure, reset
Remediation loop	experiment.py → remediate	Observe → propose → validate (→ ask a human) → act → wait; shared with on-call
Fault injection	scenarios.py → SCENARIOS	docker kill and Toxiproxy toxics; held-out flag
Recovery actions	actions.py → ACTIONS, run_action	Each action is an Ansible playbook with a 45 s timeout
Safety shield	safety.py → SafetyValidator.check	Five rules in order; needs_approval for human mode
Results storage	store.py → save, summary, trial	SQLite with additive migrations
One experiment at a time	runner.py → lab_lock	A file lock on the shared volume, across containers
CLI	*_main_*.py	list, run, train, learning-curve, report, explain, gate, oncall, drill

11.2 Learning and AI

Idea	File → function	What to say
Bandit features	bandit.py → features	8 numbers from symptoms; latencies log-scaled
LinUCB	bandit.py → LinUCB.scores, update	Per-action ridge regression plus an uncertainty bonus
Training + learning curve	learning.py → train, learning_curve	Training failures only (asserted); 30 replay orders
Policies	policies.py → RuleBased, Heuristic, Bandit, LLM, Guided*	Same inputs for all; SRE_GUIDANCE text for the ablation
OpenAI client	llm.py → chat, chat*json, chat*tools	gpt-4o-mini, temperature 0; the key never appears in errors
LLM agent	agent.py → AgentPolicy.choose, run_tool	Look before acting (1-5 tool calls); trail recorded; Toxiproxy hidden
Post-mortems, Q&A	insights.py → explain*trial, explain*incident, ask	Only measured data goes in; cached

11.3 On-call mode

Idea	File → function	What to say
Webhook receiver	oncall.py → _Webhook, serve	Alertmanager POSTs here; repeated notifications are merged
Rules of engagement	oncall.py → handle	Autopilot on? Lab free? Otherwise stand down
Confirm before acting	oncall.py → _respond	4 s of probe traffic; healthy = stale page, no incident
Working an incident	oncall.py → *work*incident	Same remediate loop; sustained recovery or escalation
Human approval	oncall.py → *ask*human; incidents.py → decide, expire	First decision wins; 60 s then fail safe
Records and scorecard	incidents.py → scorecard	Matches each fault to the agent's incident; detect / engage / recover
SLI	app/traffic/traffic.py	good = 200 within 300 ms; counters for Prometheus
SLO and alerts	observability/prometheus/rules/slo.yml	Recording rules, page and ticket alerts
Alert tests	observability/prometheus/tests/slo_test.yml	Outage pages, blip does not, slowness counts, lost SLI tickets
Operator console	dashboard/server/app.py → oncall*status, decide*approval, break*lab,* chaos_loop	SLO, approvals, Break it, monkey, rollback
On-call page	dashboard/web/src/pages/OnCall.jsx	Approval card, KPIs, timeline, controls, scorecard, history

11.4 DevOps

Idea	File	What to say
The lab	docker-compose.yml (+ override, prod, tunnel overlays)	11 services; health checks; isolated project names
Pipeline	Jenkinsfile; jenkins/jobs/chaos-os-ci/config.xml	10 stages; multibranch; polls GitHub every 5 min
Release gate	resilience-gate.yml; *_main.py* → *cmd*gate	6 faults with deadlines
On-call drill	*_main.py* → *cmd*drill	Inject unannounced; pass only if healed in time
Security scan	scripts/security-scan.sh; .gitleaks.toml; .trivyignore.yaml	Secrets, deps, Dockerfiles, images, SBOMs
Deployment	deploy/ansible/deploy.yml; deploy/caddy/Caddyfile	Provision a VM; HTTPS + login in front
Sharing	deploy/share.ps1; deploy/docker-compose.tunnel.yml	Cloudflare quick tunnel, no ports opened
Measurement campaign	scripts/oncall_campaign.py	Unannounced faults through the dashboard API; scripted operator
Paper figures	docs/paper/make*figures.py,* *make*tables.py	Every chart and table regenerated from the database

12. Appendix: the key files, annotated

These are the files teachers most often ask to see. They are copied here from the repository when this guide is built, so they are always current.

12.1 safety-policy.yml: the safety rules

Read top to bottom: approved environments, the action budget, the approval modes (auto in the lab, human on call), and for every action its impact and the cost charged in the bandit's reward.

```
# Safety policy - every proposed recovery action is checked against this
# file BEFORE it runs, whichever policy proposed it (rules, bandit, or LLM).
# The Ansible playbooks enforce their own service allowlist as a second layer.

environment:
  # Only act on approved, isolated lab projects (fnmatch patterns).
  allowed_projects: ["chaosos", "chaosos-ci-*"]

limits:
  # At most this many actions per incident; each action at most once
  # (prevents restart loops, a classic automation failure).
  max_actions_per_incident: 3

approval:
  # High-impact actions need approval:
  #   auto  = approved and logged (isolated lab experiments only)
  #   human = an operator approves or denies it in the dashboard (on-call mode)
  #   deny  = always blocked
  # Override with CHAOS_HIGH_IMPACT_APPROVAL.
  high_impact: auto
  # On-call mode handles failures nobody announced, so a human decides.
  on_call: human
  # No decision in time = not run (fail safe).
  timeout_s: 60

actions:
  # impact: none | low | medium | high
  # cost:   penalty in [0, 1] subtracted from the learning reward
  none:             {impact: none,   cost: 0.00}
  warm_cache:       {impact: low,    cost: 0.02}   # one bulk query
  cache_bypass:     {impact: low,    cost: 0.05}   # degraded mode: every read hits the
DB
  restart_redis:    {impact: medium, cost: 0.05}   # drops the whole cache
  restart_api:      {impact: medium, cost: 0.10}   # drops in-flight requests
  restart_postgres: {impact: high,   cost: 0.20}   # drops every DB connection
```

12.2 observability/prometheus/rules/slo.yml: SLO and alerts

Four recording rules compute the bad-request ratio over 20 s, 1 min, 10 min and 1 h. The page needs both the 1 min and 20 s ratios above 14.4%; the ticket needs 1 h and 10 min above 3%.

```
# Shop SLO: 99% of synthetic user requests are good (succeed within 300 ms).
# Error budget = the 1% that may be bad.
#
# Alerts use the multi-window, multi-burn-rate pattern from the Google SRE
# Workbook, scaled from hours to lab time: a page needs a long window (a real
# problem, not a blip) AND a short one (still happening, so it stops paging
# soon after recovery). Burn rate 14.4 = spending the budget 14.4x too fast.
# Tested with `promtool test rules` (observability/prometheus/tests).
groups:
```

```
  - name: shop-slo
    rules:
      - record: slo:bad_ratio:rate20s
        expr: |
          sum(rate(shop_synthetic_requests_total{result!="good"}[20s]))
            / sum(rate(shop_synthetic_requests_total[20s]))
      - record: slo:bad_ratio:rate1m
        expr: |
          sum(rate(shop_synthetic_requests_total{result!="good"}[1m]))
            / sum(rate(shop_synthetic_requests_total[1m]))
      - record: slo:bad_ratio:rate10m
        expr: |
          sum(rate(shop_synthetic_requests_total{result!="good"}[10m]))
            / sum(rate(shop_synthetic_requests_total[10m]))
      - record: slo:bad_ratio:rate1h
        expr: |
          sum(rate(shop_synthetic_requests_total{result!="good"}[1h]))
            / sum(rate(shop_synthetic_requests_total[1h]))

      # Page: wakes the on-call agent (Alertmanager webhook).
      - alert: ShopSLOFastBurn
        expr: slo:bad_ratio:rate1m > (14.4 * 0.01) and slo:bad_ratio:rate20s > (14.4 *
0.01)
        labels:
          severity: page
        annotations:
          summary: Shop is burning its error budget 14x too fast
          description: >-
            Over 14.4% of synthetic user requests failed or took longer than
            300 ms, in both the last minute and the last 20 seconds (SLO: 99% good).

      # Ticket: a slower, sustained burn worth a look, not an emergency.
      - alert: ShopSLOSlowBurn
        expr: slo:bad_ratio:rate1h > (3 * 0.01) and slo:bad_ratio:rate10m > (3 * 0.01)
        labels:
          severity: ticket
        annotations:
          summary: Shop has spent its error budget 3x too fast this hour
          description: >-
            Over 3% of synthetic user requests were bad in both the last hour and
            the last 10 minutes (SLO: 99% good).

      # Meta-monitoring: without synthetic traffic the SLO is blind.
      - alert: SyntheticTrafficDown
        expr: up{job="traffic"} == 0
        for: 30s
        labels:
          severity: ticket
        annotations:
          summary: Synthetic traffic is not being scraped
          description: The SLO cannot be measured while the traffic service is down.
```

12.3 observability/alertmanager/alertmanager.yml: routing pages to the agent

```
# Routes SLO alerts to the on-call agent (controller/chaosctl/oncall.py).
# The agent acts on severity=page and ignores tickets; it confirms every page
# with its own probe traffic, so a repeat notification is a cheap re-check.
route:
  receiver: oncall-agent
  group_by: [alertname]
  group_wait: 1s          # page immediately: the alert rules already filter blips
  group_interval: 10s
  repeat_interval: 1m     # still firing? remind the agent (it re-checks)
```

```
receivers:
  - name: oncall-agent
    webhook_configs:
      - url: http://oncall:8095/alerts
        send_resolved: true

# A fast burn already pages; don't also raise the slow-burn ticket for it.
inhibit_rules:
  - source_matchers: [alertname="ShopSLOFastBurn"]
    target_matchers: [alertname="ShopSLOSlowBurn"]
```

12.4 resilience-gate.yml: what every release must survive

```
# Resilience gate - policy as code.
# Every release must recover from each failure below, using the listed
# recovery playbook, within max_recovery_s (fault injection -> sustained
# healthy traffic). Run by Jenkins via `chaosctl gate`; any miss fails the build.
#
# Limits are roughly 2x the median measured locally, leaving headroom for a
# busier CI host.
checks:
  - scenario: redis_down
    action: restart_redis
    max_recovery_s: 10
  - scenario: redis_down
    action: cache_bypass
    max_recovery_s: 10
  - scenario: redis_latency
    action: cache_bypass
    max_recovery_s: 10
  - scenario: postgres_latency
    action: warm_cache
    max_recovery_s: 15
  - scenario: postgres_down
    action: restart_postgres
    max_recovery_s: 15
  - scenario: api_crash
    action: restart_api
    max_recovery_s: 20
```

12.5 ansible/playbooks/restart_service.yml: one recovery action

Note the first task: the playbook refuses any service outside the allowlist, even if the validator were bypassed.

```
# Recovery action: restart one lab service and wait for the API to be ready.
#   docker compose run --rm --build ansible playbooks/restart_service.yml -e
service=redis
- name: Restart a lab service
  hosts: lab
  gather_facts: false
  tasks:
    - name: Refuse services outside the safety allowlist
      ansible.builtin.assert:
        that: service in allowed_services
        fail_msg: "Service '{{ service }}' is not in allowed_services; refusing to
act."

    - name: Find containers for {{ service }}
      ansible.builtin.command: >-
        docker ps -aq
        --filter label=com.docker.compose.project={{ compose_project }}
        --filter label=com.docker.compose.service={{ service }}
```

```
      register: containers
      changed_when: false
      failed_when: containers.stdout_lines | length == 0

    - name: Restart {{ service }}
      ansible.builtin.command:
        argv: "{{ ['docker', 'restart'] + containers.stdout_lines }}"
      changed_when: true

    - name: Wait for the API to report ready
      ansible.builtin.uri:
        url: "{{ api_url }}/ready"
        status_code: 200
        timeout: 2   # a hung /ready must not stall the playbook
      register: ready
      retries: 30
      delay: 1
      until: ready.status == 200
```

12.6 Jenkinsfile: the stages

```
    stages {
        stage('Build images') {
            steps {
                sh 'docker compose --profile tools build'
            }
        }

        stage('Unit tests') {
            steps {
                sh 'docker compose run --rm --entrypoint python controller -m unittest
discover -s tests -v'
            }
        }

        stage('Alert rules') {
            steps {
                // Alerts are code too: validate the configs and unit-test the SLO
                // burn-rate rules (observability/prometheus/tests/slo_test.yml).
                sh 'docker compose run --rm --no-deps --entrypoint promtool prometheus
check config /etc/prometheus/prometheus.yml'
                sh 'docker compose run --rm --no-deps --entrypoint promtool prometheus
test rules /etc/prometheus/tests/slo_test.yml'
                sh 'docker compose run --rm --no-deps --entrypoint amtool alertmanager
check-config /etc/alertmanager/alertmanager.yml'
            }
        }

        stage('Security scan') {
            steps {
                // Secrets in code and git history (gitleaks), vulnerable dependencies
and
                // Dockerfile misconfigurations (trivy fs), image CVEs and SBOMs (trivy
image).
                sh 'sh scripts/security-scan.sh'
            }
            post {
                always {
                    archiveArtifacts artifacts: 'security-reports/**',
allowEmptyArchive: true
                }
            }
        }
```

```
        stage('Deploy lab') {
            steps {
                sh 'docker compose up -d --wait'
                sh 'docker compose ps'
            }
        }

        stage('Smoke test') {
            steps {
                // Piped via stdin so no bind mount is needed (Docker-outside-of-
Docker).
                sh 'docker run --rm -i --network "$LAB_NETWORK" python:3.12-slim python
- http://frontend:8080 < scripts/smoke_test.py'
            }
        }

        stage('Resilience gate') {
            steps {
                // Checks and limits live in resilience-gate.yml (policy as code).
                sh 'docker compose run --rm controller gate'
            }
            post {
                always {
                    sh 'docker compose run --rm -T controller report > resilience-
report.txt || true'
                    sh 'cat resilience-report.txt || true'
                    archiveArtifacts artifacts: 'resilience-report.txt',
allowEmptyArchive: true
                }
            }
        }

        stage('On-call drill') {
            steps {
                // Break the lab without telling anyone: the SLO alert must page the
                // on-call agent, and the agent must heal it on its own in time.
                // (no pipe to tee: dash has no pipefail, so a failed drill would look
green)
                sh '''
                    rc=0
                    docker compose run --rm -T controller drill --scenario redis_down -
-within 120 > oncall-drill.txt 2>&1 || rc=$?
                    cat oncall-drill.txt
                    exit $rc
                '''
            }
            post {
                always {
                    archiveArtifacts artifacts: 'oncall-drill.txt', allowEmptyArchive:
true
                }
            }
        }
    }

    post {
        always {
            sh 'docker compose logs --no-color > lab-logs.txt || true'
            archiveArtifacts artifacts: 'lab-logs.txt', allowEmptyArchive: true
            sh 'docker compose --profile tools down -v --remove-orphans || true'
        }
    ... (see the file for the rest)
```

12.7 .trivyignore.yaml: accepted security risks, each with a reason

```
# Accepted risks for Trivy (scripts/security-scan.sh). Every entry states why
# it is acceptable here; revisit them when a base image or the design changes.
vulnerabilities:
  - id: CVE-2025-68121
    paths:
      - usr/local/bin/gosu
    statement: >-
      Go standard-library CVE in gosu, which the official postgres image runs once
      at start-up to drop root before starting the server. gosu handles no network
      or untrusted input, so the vulnerable code is not reachable. Re-check when the
      official image ships a gosu built with a newer Go.
    expired_at: 2027-03-01

misconfigurations:
  - id: AVD-DS-0002
    paths:
      - controller/Dockerfile
      - dashboard/Dockerfile
      - ansible/Dockerfile
    statement: >-
      These run the Docker CLI against the host's /var/run/docker.sock to inject
      faults and restart lab containers, which needs root in the container.
      Mitigations: no published ports in production (only Caddy, behind a login),
      every action passes the safety validator, and the playbooks enforce their
      own service allowlist.
  - id: AVD-DS-0002
    paths:
      - app/db/Dockerfile
    statement: >-
      The official postgres entrypoint starts as root only to set data-directory
      ownership, then drops to the postgres user for the server itself.
  - id: AVD-DS-0002
    paths:
      - deploy/runner/Dockerfile
    statement: >-
      One-shot deployment tool that runs on the operator's own machine; it is
      never deployed or exposed.
  - id: AVD-DS-0031
    paths:
      - deploy/runner/Dockerfile
    statement: >-
      False positive: ANSIBLE_HOST_KEY_CHECKING is a boolean setting whose name
      contains "KEY"; no secret is passed through the environment.
```